

Tribhuvan University
Bhaktapur Multiple Campus



A Final Year Report
On
TEXT SUMMARIZATION USING LSA AND T5 TRANSFORMER

Under the Supervision of
Mr. Umesh Dahal
Lecturer, Bhaktapur Multiple Campus

Submitted To:
Department of Computer Science and Information Technology
Bhaktapur Multiple Campus
Dudhpati, Bhaktapur

In partial fulfillment of the requirement for the Bachelor Degree in
Computer Science and Information Technology

Submitted By :
Tilak Parajuli (23282/076)
Swechha Shrestha (23281/076)
Suman khadka (23275/076)

April 28th, 2024



SUPERVISOR'S RECOMMENDATION

I hereby recommend that the report prepared under my supervision by Tilak Parajuli (TU Exam Roll No. 23282/076), Swechha Shrestha (TU Exam Roll No. 23281/076), Suman Khadka (TU Exam Roll No. 23275/076) entitled “**TEXT SUMMARIZATION USING LSA AND T5 TRANSFORMER**” submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Information Technology, Tribhuvan University be processed for evaluation.

Mr. Umesh Dahal

Lecturer,

Department of CSIT

Bhaktapur Multiple Campus

Dudhpati, Bhaktapur



CERTIFICATE OF APPROVAL

This is to certify that this project prepared by Tilak Parajuli (TU Exam Roll No. 23282/076), Swechha Shrestha (TU Exam Roll No. 23281/076), Suman Khadka (TU Exam Roll No. 23275/076) entitled “**TEXT SUMMARIZATION USING LSA AND T5 TRANSFORMER**” in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science and Information Technology has been well studied. In our opinion, it is satisfactory in the scope and quality as a project for the required degree.

Mr. Umesh Dahal

Project Supervisor

Lecturer,

Bhaktapur Multiple Campus

Dudhpati, Bhaktapur

Mr. Sushant Poudel

Coordinator, Department of CSIT

Bhaktapur Multiple Campus

Dudhpati, Bhaktapur

Mr.

External Examiner

ACKNOWLEDGEMENT

In our pursuit of goals and dreams, heartfelt gratitude is extended to those instrumental in making this journey possible. Foremost, we express sincere appreciation to our families—the steadfast bedrock of our endeavors, whose sacrifices, both direct and indirect, have been indispensable.

A special acknowledgement is reserved for Mr. Umesh Dahal, whose unwavering support and encouragement have propelled our project. His patience, friendliness, and profound subject knowledge significantly contributed to our achievements. Mr. Dahal's consistent presence in project meetings and his effective problem-solving approach have been invaluable.

We are grateful to the Department of Computer Science and Information Technology at Bhaktapur Multiple Campus for providing an exceptional working environment, and we extend our thanks to the friendly and encouraging mentors who guided us throughout the project.

Special recognition is given to the Head of the Department, Mr. Sushant Poudel, the driving force behind our project completion. His dedicated support, both technical and non-technical, and guidance across various IT sectors played a pivotal role in our success. We consider ourselves fortunate to have someone so committed to the well-being of students.

Collectively, gratitude is extended to faculty members, friends, family, well-wishers, juniors, and seniors who were integral to our project. Their constructive suggestions, criticisms, and feedback enriched our experience, filling our journey with awe and reverence.

Sincerely,

Tilak Parajuli (TU Exam Roll No. 23282/076)

Swechha Shrestha (TU Exam Roll No. 23281/076)

Suman Khadka (TU Exam Roll No. 23275/076)

ABSTRACT

In response to the ever-growing volume of medical research papers, this project investigates the effectiveness of two distinct approaches – Latent Semantic Analysis (LSA) and fine-tuned T5 Transformer – for text summarization in the medical domain. LSA, known for uncovering latent semantic structures, is employed alongside fine-tuned T5 Transformer, a state-of-the-art deep learning architecture tailored to medical research literature. The goal of this system is to extract crucial information from complex medical texts, providing users with a concise yet comprehensive understanding of key insights. This project addresses the unique challenges in summarizing medical research papers, where specialized attention to domain-specific terminology is crucial. By leveraging both LSA and fine-tuned T5 Transformer separately, the system enhances summarization accuracy and ensures a focus on critical medical terminology. Evaluation of the system involves rigorous assessments, considering metrics such as Rouge scores, Recall, Precision, and the F-score, to measure the quality and effectiveness of the generated summaries. The project outcomes aim to significantly assist medical researchers in efficiently extracting core findings from extensive research papers, thereby fostering a more informed research environment.

Keywords: LSA, Fine-tuning, Summarization, Text processing, Evaluation metrics, T5 Transformer, TF-IDF, TDM, DTM

TABLE OF CONTENTS

SUPERVISOR’S RECOMMENDATION	i
CERTIFICATE OF APPROVAL	ii
ACKNOWLEDGEMENT.....	iv
ABSTRACT.....	v
TABLE OF CONTENTS	vi
LIST OF FIGURES	viii
LIST OF TABLES.....	ix
LIST OF ABBREVIATIONS	x
CHAPTER 1: INTRODUCTION.....	1
1.1 Project Introduction	1
1.2 Problem Statement.....	2
1.3 Objectives	3
1.4 Scope and Limitations.....	3
1.4.1 Scope.....	3
1.4.2 Limitations	3
1.5 Development Methodology	4
1.5 Report Organization.....	7
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW	8
2.1 Background Study.....	8
2.1.1 Text Summarization Approaches.....	8
2.2 Literature Review.....	10
CHAPTER 3: SYSTEM ANALYSIS	12
3.1 System Analysis.....	12
3.1.1. Requirement Analysis.....	12
3.1.2 Feasibility Analysis.....	14

3.1.3 Analysis.....	16
CHAPTER 4: SYSTEM DESIGN	22
4.1 Design	22
4.1.1 Component Diagram.....	23
4.1.2 Deployment Diagram.....	24
4.2 Algorithm Details.....	25
4.2.1 Algorithm for LSA.....	25
4.2.2 Algorithm for Text Summarization using T5 Transformer	27
4.2.3 Algorithm for Topic Extractor	30
4.2.4 Metrics used	30
CHAPTER 5: IMPLEMENTATION AND TESTING	32
5.1 Implementation	32
5.1.1 Tools Used	32
5.1.2 Implementation Details of LSA Algorithm	33
5.1.3 Implementation Details of T5 Algorithm	38
5.1.3 Implementation Details of Topic Extraction.....	41
5.2. Testing.....	44
5.2.1. Test Cases for Unit Testing.....	44
5.2.2. Test Cases for System Testing.....	46
5.3. Result Analysis	47
CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATION.....	53
6.1 Conclusion	53
6.2 Future Recommendation.....	53
REFERENCES.....	55
APPENDICES	58

LIST OF FIGURES

Figure 1.1: Global Healthcare Data Growth (2013-2020).	1
Figure 1.2: Agile Development Cycle	5
Figure 1.3: Agile Workflow Iteration	6
Figure 2.1: Extractive Summarizer	9
Figure 2.2: Abstractive Summarizer	9
Figure 3.1: Use case diagram.....	13
Figure 3.2: Gantt Chart	16
Figure 3.3: Dataset format	17
Figure 3.4: Class Diagram for text summarization system.....	18
Figure 3.5: Sequence Diagram for Text summarization system.....	19
Figure 3.6: Activity Diagram for Text summarization system	21
Figure 4.1: Block diagram of Text summarization system.....	22
Figure 4.2: Component Diagram for Text summarization system	23
Figure 4.3: Deployment Diagram for Text summarization system	24
Figure 5.1: Working mechanism of LSA Algorithm	37
Figure 5.2: Block diagram for Fine-tuning using T5 transformer	41
Figure 5.3: Summary generated by LSA for sample text	47
Figure 5.4: Rouge score for the summary generated by LSA for sample text.....	48
Figure 5.5: Summary generated by fine-tuned T5 model for sample text	48
Figure 5.6: Rouge score for the summary generated by T5 for sample text.....	49
Figure 5.7: Rouge Score Comparison of LSA and T5.....	50
Figure 5.8: Training Loss over Epochs.....	51
Figure 5.9: Comparison between actual and predicted summary length	51

LIST OF TABLES

Table 3.1: Project Schedule	16
Table 4.1: HyperParameter	29
Table 5.1: User Registration Text Case	44
Table 5.2: User Login Test Case.....	45
Table 5.3: Input Text Validation Test Case	45
Table 5.4: System Test Case	46
Table 5.5: Rouge Scores for LSA and T5.....	49

LIST OF ABBREVIATIONS

AI: Artificial Intelligence

DTM: Document Term Matrix

LSA: Latent Semantic Analysis

NLP: Natural Language Processing

ROUGE: Recall Oriented Understudy for Gisting Evaluation

SVD: Singular Value Decomposition

T5: Text To Text Transfer Transformer

TDM: Term Document Matrix

TF-IDF: Term Frequency-Inverse Document Frequency

CHAPTER 1: INTRODUCTION

1.1 Project Introduction

As the internet expands and the volume of information and documents grows exponentially, individuals find themselves overwhelmed with vast amounts of data. This challenge is particularly pronounced in specialized domains such as medical research papers, where the need for efficient information extraction is crucial. This has prompted researchers to seek technological solutions for automatic text summarization, aiming to condense and simplify the wealth of textual content available. The process of automatic text summarization involves generating a brief and coherent summary while maintaining essential information and the overall significance of the original text [1]. So the original intent of the text or document is preserved.

The escalating volume of scientific publications, particularly in the realm of medical research, poses a significant challenge for researchers and practitioners seeking to stay abreast of the latest developments. The unprecedented global health crisis triggered by the COVID-19 pandemic has exemplified this issue, with over 125,000 COVID-19-related scientific articles released within a mere 10 months of the first confirmed case [2].

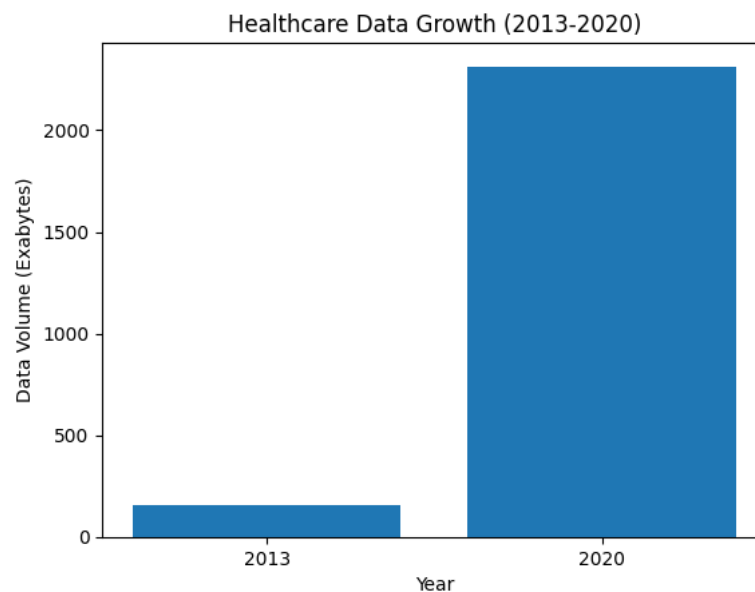


Figure 1.1: Global Healthcare Data Growth (2013-2020). [9]

Platforms like PubMed [9] serve as critical repositories for vast amounts of biomedical data, hosting millions of records. Navigating through this extensive dataset manually is an impractical task due to its sheer volume. The abundance of information on platforms like PubMed underscores the need for advanced tools and technologies, such as text summarization systems, to efficiently extract essential insights and provide concise summaries. These tools play a pivotal role in aiding researchers, clinicians, and healthcare professionals in swiftly accessing relevant information amidst the overwhelming volume of data available on biomedical databases.

Automatic summarization, broadly categorized into extractive and abstractive methods, provides an efficient means to distill crucial insights from extensive datasets. Extractive methods involve selecting and combining existing sentences, while abstractive methods generate new, concise content that encapsulates the essence of the original text. In the domain of biomedical research, where specialized terminology is abundant, the need for domain-specific summarization becomes even more pronounced. Traditional methods often fall short in capturing the intricacies of medical literature, emphasizing the importance of leveraging advanced techniques, such as domain-specific abstractive summarization, to enhance information extraction in the biomedical domain.

1.2 Problem Statement

Medical research papers are rich sources of knowledge, but their dense language and specialized terminology pose a challenge for efficient comprehension. Manually summarizing key findings while retaining vital medical details is time-consuming and resource-intensive for researchers.

Traditional extractive summarization methods often excel in generating factual summaries but may fall short in providing a deep understanding and context of the underlying information. Existing automatic summarization methods, such as standard Transformers, often struggle with capturing domain-specific medical terminology and contexts. While they may generate summaries, they lack focus on the critical medical details researchers need. Focusing solely on generic summarization techniques can lead to summaries that miss crucial medical insights. In response to these challenges, our proposed system combines the strengths of domain-specific fine-tuning and latent

semantic analysis. By leveraging both techniques independently, our system aims to provide researchers with accurate summaries of medical articles.

1.3 Objectives

The main objectives of the project are:

- Address the challenge of summarizing medical research papers by developing a system that takes into account the domain-specific terminology.
- Provide users with the ability to choose between LSA and the fine-tuned T5 transformer.

1.4 Scope and Limitations

1.4.1 Scope

The scope of the project is to generate a text summarizer that provides a concise summary by identifying key topics and relevant information. The following are the scope of this project:

- The project aims to offer users a choice between two distinct summarization algorithms, Latent Semantic Analysis (LSA) and a T5 Transformer fine-tuned on medical research papers.
- The primary focus is on the summarization of medical research papers, incorporating domain-specific terminology.
- The system will provide a user-friendly interface, allowing users to easily upload medical research papers and select their preferred summarization algorithm, fostering interactive user experience.
- The project involves the implementation and evaluation of performance metrics to objectively measure the effectiveness of the summarization algorithms, including accuracy, coherence, and relevance of the generated summaries.

1.4.2 Limitations

The project has following limitations:

- While the system might offer improved performance for medical summarization, certain limitations may still exist, such as challenges in handling highly technical or ambiguous medical terms.

- The system's performance may not fully replicate the understanding of a human expert in the medical field. It is designed to aid researchers in obtaining key information but may not replace the need for human expertise in complex medical analysis.

1.5 Development Methodology

For our project, we have adopted the Agile methodology [16], renowned for its adaptability and responsiveness to evolving project requirements. This methodology, symbolized by its iterative development approach, aligns seamlessly with the dynamic characteristics of our project, providing a flexible and collaborative environment throughout the development lifecycle.

The methodology unfolds through structured sprint planning sessions, where tasks are meticulously defined, goals are set, and responsibilities are allocated for each iteration. This practice establishes a transparent roadmap, facilitating efficient progress tracking and ensuring alignment with project objectives.

During each sprint, active development takes center stage, emphasizing collaboration among team members. The integration of Google Colab enhances this collaborative effort, enabling real-time coding synchronization. Simultaneously, continuous testing protocols swiftly identify and address potential issues, preserving the integrity of the codebase.

At the end of each sprint, a comprehensive review showcases completed work and gathers constructive feedback. Retrospectives follow, providing the team with an opportunity to reflect on processes, identify improvement areas, and refine the development approach.

Effective collaboration and communication, pivotal in Agile, are facilitated through Google Workspace tools—Docs and Sheets—allowing real-time collaboration on project documentation. Team communication is streamlined via Google Space, ensuring instant messaging, file sharing, and efficient meeting scheduling.

Project tracking is meticulously managed through Trello and a shared Google Sheets document, encompassing task statuses, team assignments, and other relevant project metrics. Concurrently, Google Calendar is employed for seamless scheduling of meetings, deadlines, and other project-related events.

Agile's proactive risk management involves identifying potential risks at the sprint outset, followed by strategic mitigation discussions. Stakeholder involvement during sprint reviews ensures their valuable input guides the project in alignment with overarching goals.

Finally, the inherent adaptability of the Agile methodology allows for adjusting sprint goals based on feedback and evolving project requirements. This flexibility ensures a dynamic and responsive development approach tailored to the academic context of our college project.

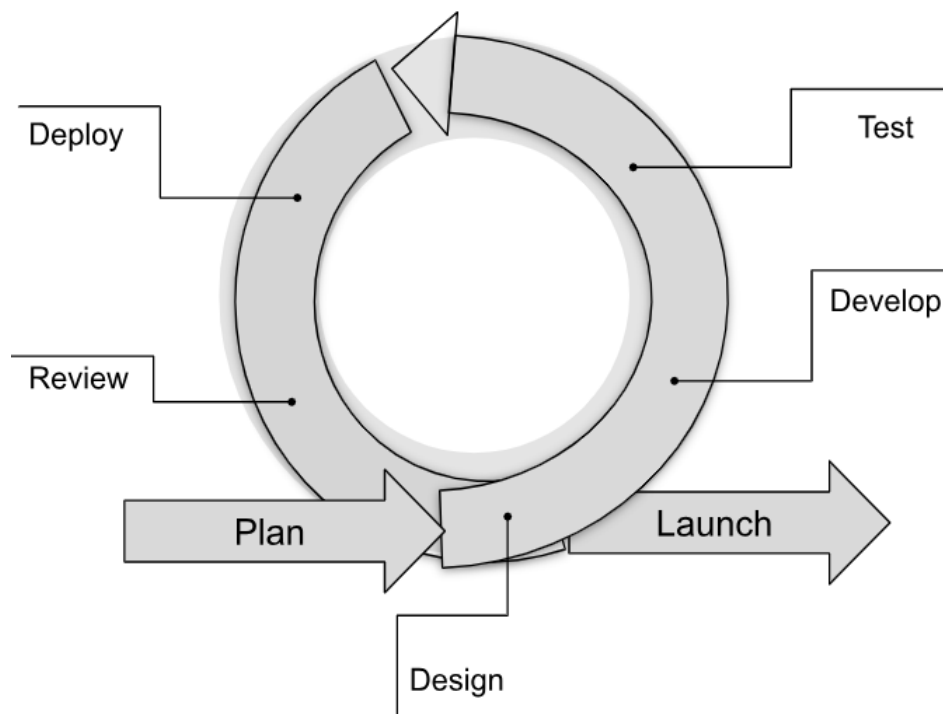


Figure 1.2: Agile Development Cycle

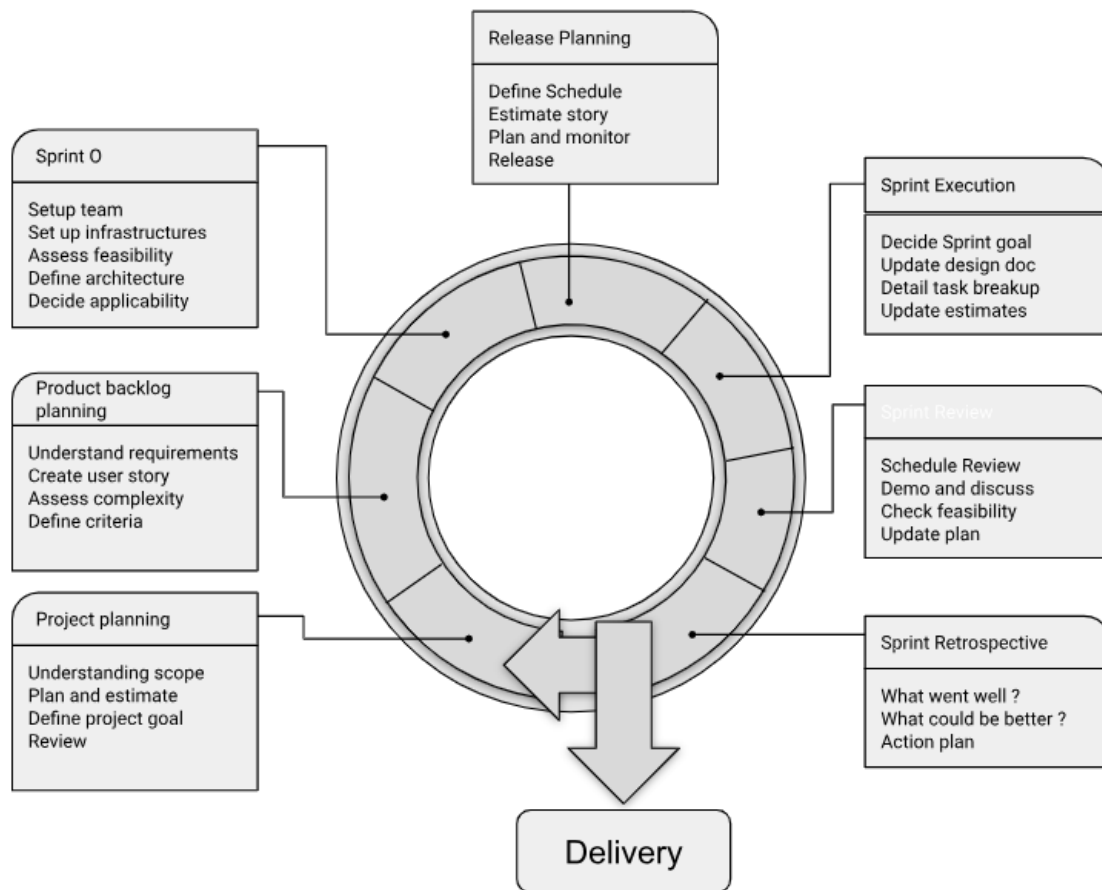


Figure 1.3: Agile Workflow Iteration

This integrated approach not only ensures the successful completion of project goals but also aligns with educational principles, promoting active learning, collaboration, adaptability, continuous improvement, and effective communication within our academic context.

1.5 Report Organization

The report is organized as follows:

Chapter 1: Introduction

Projects are introduced, with their objectives, scope and limitations.

Chapter 2: System Analysis

Includes reviews of the past literature based on similar projects. Emphasizes the different functional and non-functional requirements of the system, and the detailed description of the dataset used in the project. Feasibility analysis and process modeling are done to analyze the system's feasibility and working mechanism.

Chapter 3: System Design

This chapter is concerned about the elaborated design of the system, data used, and implementation process used throughout the project.

Chapter 4: Implementation

It includes software tools, dependencies and hardware tools used to implement the system. This describes how all the steps in the process model were implemented.

Chapter 5: Testing

This chapter consists of different levels of tests carried out to test the model built for text summarization.

Chapter 6: Conclusion

This chapter includes the report conclusion and the further work that can be done concerning the enhancement of the project.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Traditional methods of information retrieval and text analysis often prove inadequate in meeting the needs of medical professionals. Extractive summarization, involving the selection of sentences or phrases from the original text, can yield fragmented summaries lacking the coherent context crucial for genuine comprehension. This limitation impedes the ability of healthcare professionals, researchers, and students to swiftly extract essential insights buried within dense medical documents.

In response to these challenges, abstractive summarization emerges as a more advanced technique in natural language processing [3]. Unlike extractive methods, abstractive summarization doesn't merely copy and paste segments of text; instead, it generates summaries in a human-like manner by understanding the underlying meaning of the content. This innovative approach holds the potential to revolutionize how medical professionals engage with literature, enabling swift access to concise, coherent, and informative summaries.

The ascent of transformer-based models, exemplified by the success of models like BERT and GPT, has propelled abstractive summarization to new heights. These models excel in understanding contextual relationships and generating coherent text, making them ideal candidates for crafting accurate and meaningful medical summaries.

However, the medical domain's intricacies, characterized by specialized terminologies, contexts, and critical nuances, present unique challenges. A successful abstractive summarization system for medical texts must capture not only factual content but also contextual intricacies essential for accurate interpretation.

2.1.1 Text Summarization Approaches

Automatic Text Summarization (ATS) has become a buzzing research area in Natural Language Processing (NLP). In simple terms, ATS involves creating a shorter version of a given text that captures its key information automatically. When it comes to categorizing ATS systems, there are a few key distinctions to consider:

1. Extractive vs. Abstractive:

Extractive approaches pick out the most relevant sentences from the source text and combine them to form a summary. Abstractive approaches, on the other hand, rephrase and paraphrase the content of the source text to generate a concise summary [4].

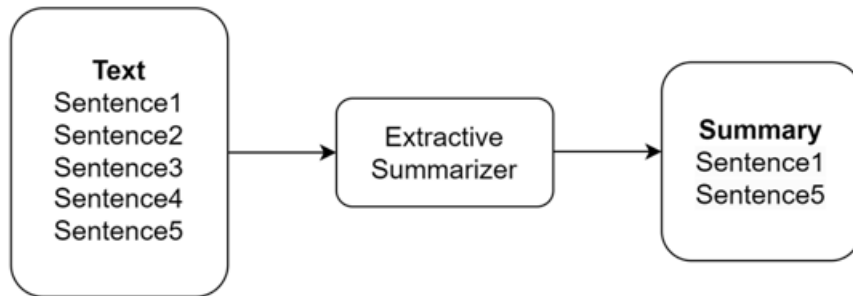


Figure 2.1: Extractive Summarizer

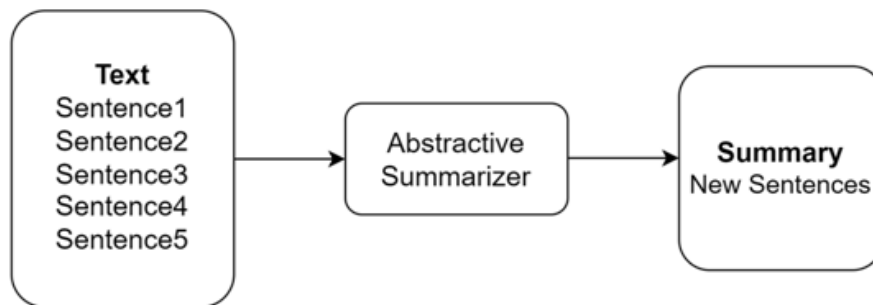


Figure 2.2: Abstractive Summarizer

2. Mono-document vs. Multi-document:

Mono-document approaches focus on summarizing one text at a time. Multi-document approaches, however, aim to summarize content from multiple documents in a single brief paragraph [16].

3. Generic vs. Query-based:

Generic summaries pull together information from the entire document to create a condensed version. Query-based summaries, on the contrary, address specific questions related to the document content [17].

2.2 Literature Review

Over the years, researchers have developed several techniques to get automatic summaries. The first approaches worked on extractive summarization, assuming that the most important words are repeated most frequently. Hans Peter Luhn pioneered automatic summarization in 1958 with his work on 'The automatic creation of literature abstracts' [10], introducing a heuristic method based on statistical word frequencies to determine sentence importance. He assumed that the most important words in a summary are repeated most frequently in the source text. He based his research on $TF - IDF$.

The foundational aspect of many text summarization approaches involves the extraction of significant sentences or phrases from the original text. This paper [3] introduces TextRank, a seminal unsupervised graph-based algorithm for extractive summarization. In TextRank, sentences are represented as nodes in a graph, and the algorithm assigns importance scores to sentences based on their relationships within the document. The scoring mechanism, similar to the PageRank algorithm, identifies sentences central to the document's meaning, enabling the extraction of key information for summarization.

Another approach to extractive summarization, Latent Semantic Analysis (LSA) explicitly aims to capture semantic relationships between words and sentences [8]. This study [5] introduces Latent Semantic Analysis as a method for producing well-defined summaries. Singular value decomposition plays a pivotal role in extracting essential sentences, ranking each sentence based on its importance in the original document. This study evaluates summaries using Rouge scores, with a specific emphasis on Recall, Precision, and the F-score.

Similarly, this paper [11] mentions three approaches for sentence extraction, namely, a frequency-based approach, a feature-based approach, and a machine learning-based approach. The exploration of these approaches provides additional insights into the diverse strategies employed in the domain of extractive summarization.

Deep Neural Networks (DNNs) made it possible to generate abstractive summaries with the use of sequence-to-sequence models, such as Recurrent Neural Networks (RNNs) and Long Short Term Memory (LSTM) [13]. SummaRuNNer [18] presents a Recurrent Neural Network based sequence model tailored specifically for extractive summarization

tasks. In tests, it showcased impressive performance, effectively selecting salient sentences from documents with high accuracy. Transformers [6], a novel neural network based on the attention mechanism, outperformed state-of-the-art approaches released before 2017 and inspired several language models usable for automatic summarization, such as BERT (Bidirectional Encoder Representations from Transformers) [15], and T5 [7]. Large language models (LLMs) like Chat Generative Pretrained Transformer (ChatGPT) have also shown potential for summarizing text, including the possibility of summarizing medical literature [19]. Fine-tuning these models on specific summarization tasks can further enhance their performance and adapt them to domain-specific requirements.

T5 [7], or Text-To-Text Transfer Transformer, is a versatile model that frames all tasks as text-to-text transformations and offers a unified approach that simplifies model training. T5 employs a massive encoder-decoder architecture pre-trained on a colossal dataset of text and code. This pre-training allows T5 to be fine-tuned for many tasks, including summarization. T5 achieves state-of-the-art results in summarization tasks, showcasing its effectiveness across various natural language processing tasks and its adaptability through fine-tuning for specific domains or tasks.

CHAPTER 3: SYSTEM ANALYSIS

3.1 System Analysis

Seeing the possibility of frequent changes that may occur during the development, an Agile model was used to develop the application. The detailed methodologies used to develop the application are described in the following subsections.

3.1.1. Requirement Analysis

The following is a detailed analysis of the functional and non-functional requirements of the project:

i. Functional Requirements

- The system should allow users to choose between two summarization algorithms: Latent Semantic Analysis (LSA) and a T5 Transformer fine-tuned on medical research papers.
- Users should be able to upload medical research papers or input text documents to be summarized.
- Summaries should be presented in a readable and coherent format, allowing users to quickly comprehend the key points of the medical research papers.
- The system should incorporate a mechanism to evaluate the effectiveness of the selected summarization algorithm, possibly through metrics like precision, recall, and F-score.
- Users should have the option to customize summarization settings, such as the length of the generated summary.

ii. Non-Functional Requirements

- The system should provide timely responses, ensuring efficient summarization even for lengthy medical research papers.
- Summaries generated by the system should maintain a high level of accuracy, reflecting the essential information present in the medical research papers.
- The user interface should be intuitive and user-friendly, providing a positive and seamless experience for users with varying levels of technical expertise.

- The system should be adaptable to future advancements in summarization algorithms or additional features that may enhance its functionality.

Use case Diagram

A use case diagram visually represents the different ways users can interact with the system. It outlines the specific use cases, or functionalities, that users can perform within the system. Actors, representing different user roles, are connected to these use cases, illustrating the interactions between users and the system. This diagram provides a simplified overview of the system's functionalities and the actors involved in utilizing those functionalities.

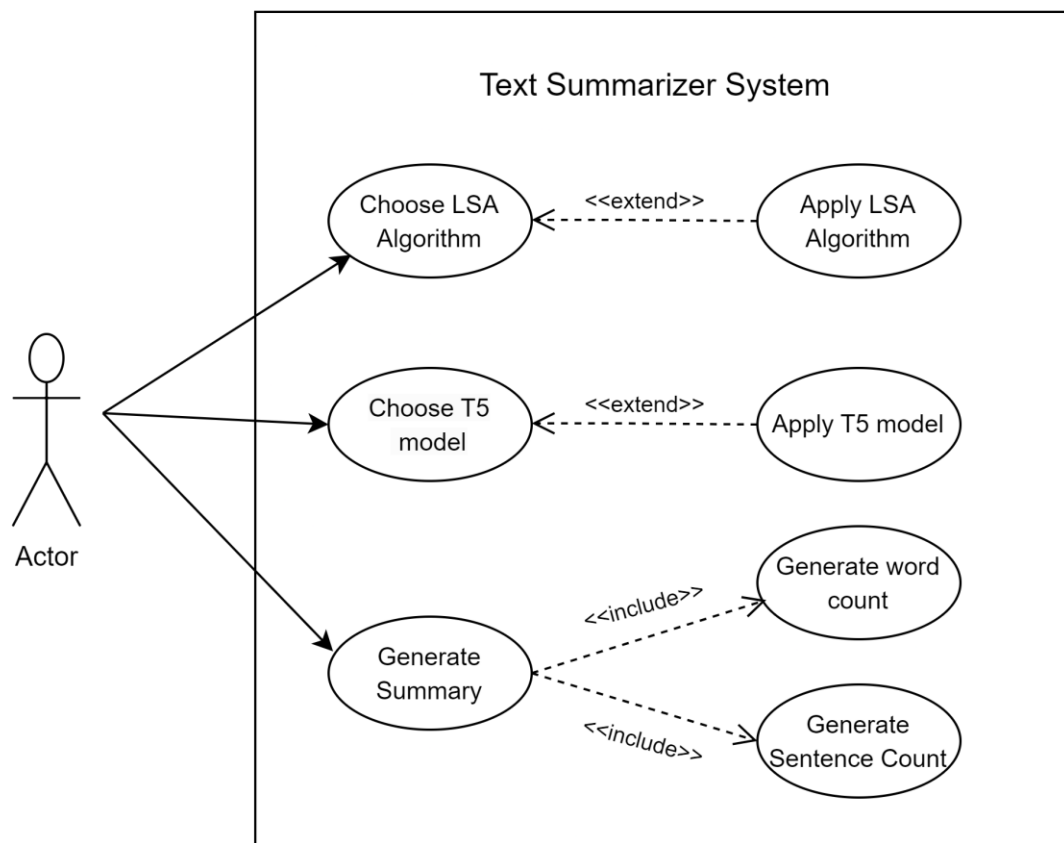


Figure 3.1: Use case diagram

3.1.2 Feasibility Analysis

i. Technical

The technical feasibility of the summarization project is high. All of the necessary technologies and resources are available to develop and deploy the system. The following are some of the key technologies that will be used to develop the summarization tool:

- **Transformers:** Transformers are a state-of-the-art AI architecture that has been shown to be highly effective for natural language processing (NLP) tasks. Transformers will be used to develop the AI model that will extract key information from input text and generate summaries.
- **LSA Implementation:** Latent Semantic Analysis (LSA) is a widely studied and implemented technique in the field of natural language processing. This ensures the availability of well-documented and efficient implementations in the chosen programming language, such as Python.
- **Django Framework:** Django is a robust and well-documented web framework for Python. The system will utilize Django to create a web application that interacts with the LSA and transformer based text summarization backend.

The team has the necessary technical expertise to develop the system. The team comprises passionate students with shared enthusiasm for AI, NLP, and Deep Learning. Overall, the technical feasibility of the project is high.

ii. Operational

The summarization project is operationally feasible. The necessary resources and support systems are in place to successfully implement and deploy the tool. The team has developed a comprehensive operational plan that outlines the steps that will be taken to implement and deploy the tool. The plan includes the following:

- The proposed project aims to provide a straightforward text summarization solution using Latent Semantic Analysis and fine-tuned T5 transformer. With a user-friendly interface, it will be designed for simplicity and ease of use, ensuring accessibility for a diverse user base.

- By automating the summarization process, the tool aims to enhance user workflows, providing a quick and efficient way to obtain condensed and meaningful summaries of text.
- The project will include thorough documentation, including code comments for developers and a user guide for end-users. This documentation will serve as a valuable resource for troubleshooting and understanding the system.

The user-friendly interface, minimal training requirements, and integration with existing workflows make it an accessible and practical tool for users with varying levels of technical expertise. Thorough documentation and potential online support channels will further contribute to the project's operational success.

iii. Economic

The text summarization project is economically feasible, supported by the utilization of open-source tools and libraries, mitigating the need for costly software licenses. Leveraging such resources significantly reduces the expenses associated with software development. The project's reliance on freely available resources further contributes to cost savings. The economic feasibility is enhanced by the moderate computational requirements for LSA implementation, eliminating the necessity for significant investments in specialized hardware or infrastructure. Additionally, the fine-tuning of the T5 model can be achieved efficiently, and if necessary, cloud services may be employed with minimal economic impact, as many providers offer free tiers or educational credits.

iv. Schedule

The project aligns with the academic schedule, ensuring optimal time allocation for student contributors. The timely completion allows for the incorporation of feedback from project advisors and mentors throughout the development phases. This iterative approach ensures continuous improvement and adherence to project goals.

Table 3.1: Project Schedule

S.N.	Task	Duration	Start Date	End Date
1	Define project scope and objectives	1 week	12 Oct 2023	18 Oct 2023
2	Literature Review	3 week	19 Oct 2023	8 Nov 2023
3	Proposal preparation	1 week	9 Nov 2023	16 Nov 2023
4	Requirement Gathering	1 week	17 Nov 2023	24 Nov 2023
5	System Design	1 week	25 Nov 2023	3 Dec 2023
6	System Development	3 week	4 Dec 2023	1 Jan 2023
7	Feedback and Testing	1 week	2 Jan 2024	10 Jan 2024
8	Documentation	1 week	11 Jan 2024	18 Jan 2024

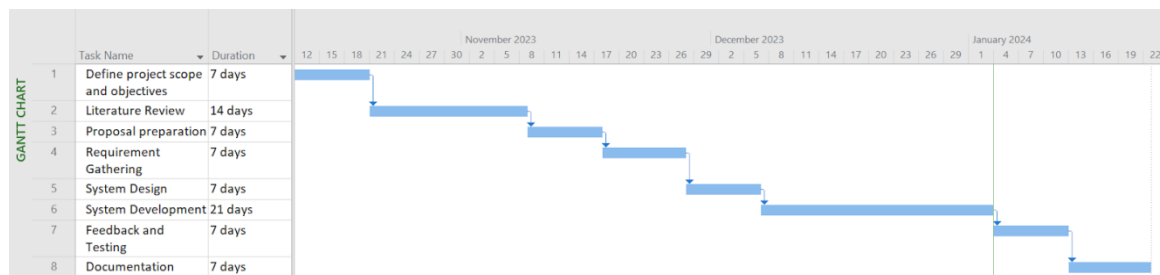


Figure 3.2: Gantt Chart

3.1.3 Analysis

The project follows object oriented approach to system analysis. Object-oriented analysis allowed us to model complex relationships and offers flexibility and scalability.

3.1.3.1 Data Collection

The PubMed Summarization Dataset, available through Hugging Face, provides an extensive collection of scientific articles directly sourced from PubMed. This dataset comprises multiple files, including validation.csv, train.csv, and test.csv, each containing a mix of articles and corresponding abstracts. With a total of 133k rows across these files, the dataset offers a diverse array of scientific topics spanning various disciplines. During the data collection process, we selected a portion of the dataset, comprising 35000 rows, for training purposes. This structured dataset, featuring the full text of articles alongside their summarized abstracts, serves as a valuable resource for tasks such as text summarization [20].

	article	abstract
0	venous thromboembolism (vte) comprising of d...	background and aim : there is lack of substant...
1	there is an epidemic of stroke in low and midd...	backgroundthe questionnaire for verifying stro...
2	cardiovascular diseases account for the highes...	background : timely access to cardiovascular h...
3	results of a liquid culturing system (bd bact...	to determine differences in the ability of myc...
4	the need for magnetic resonance imaging (mri ...	aimsour aim was to evaluate the potential for ...

Figure 3.3: Dataset format

3.1.3.2 Object modeling using Class Diagram

Object modeling using Class Diagram involves visually representing the structure and relationships of classes within a software system, facilitating a clear understanding of how data and behavior are organized and interconnected.

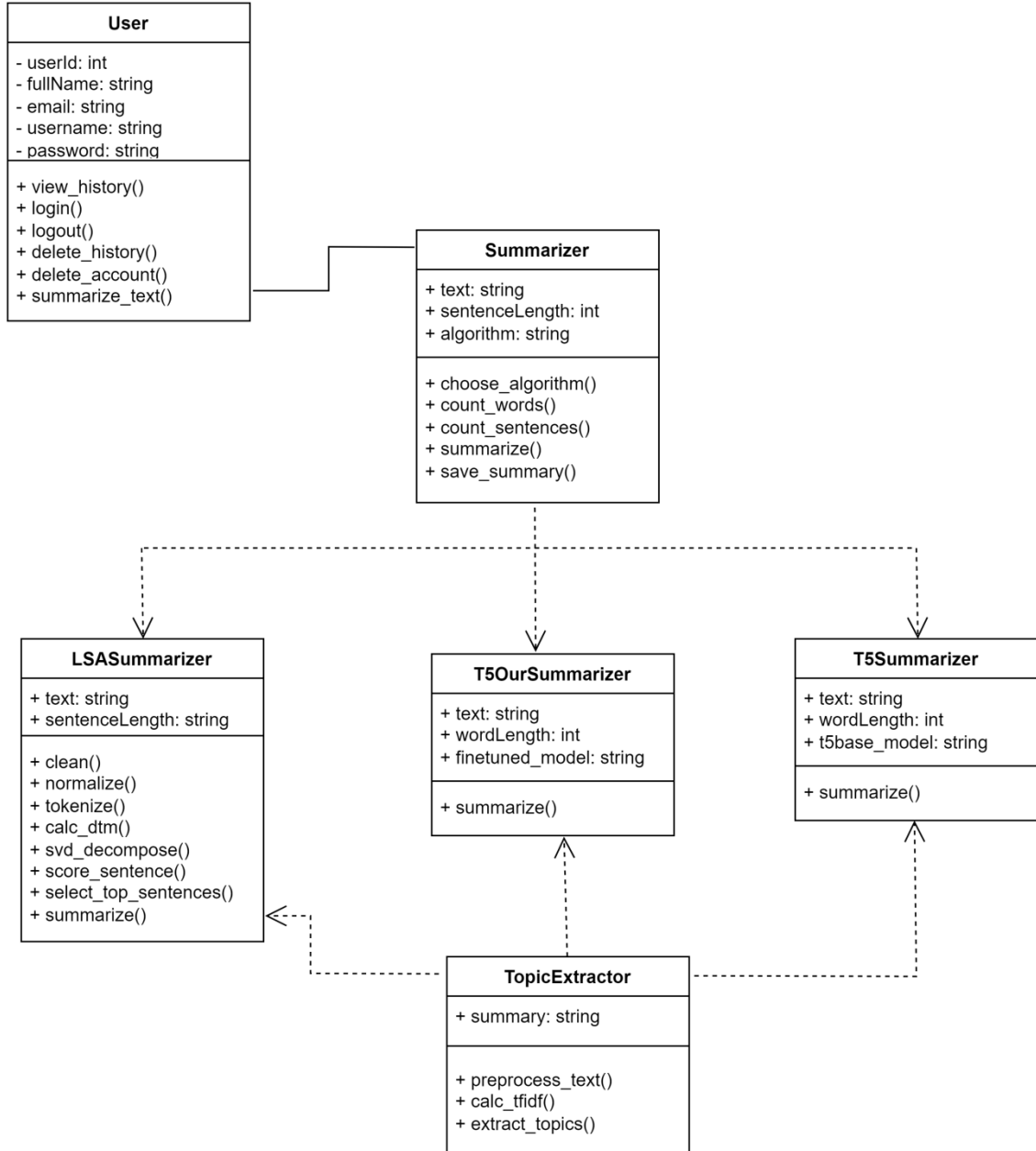


Figure 3.4: Class Diagram for text summarization system

The Figure 3.3 depicts the key classes and their relationships. User represents a system user who can log in and utilize the text summarization functionality. The class handles user login credentials and ensures only authorized users can access summarization features. The Summarizer carries out the core text summarization process. It interacts with other classes to obtain user-selected algorithms and perform the summarization task.

3.1.3.3 Dynamic modeling using Sequence Diagram

Sequence Diagram is a type of diagram that illustrates the interactions and message flows between objects or components over time, providing insights into the runtime behavior and sequence of operations within a system. It helps visualize the chronological order of method invocations and responses.

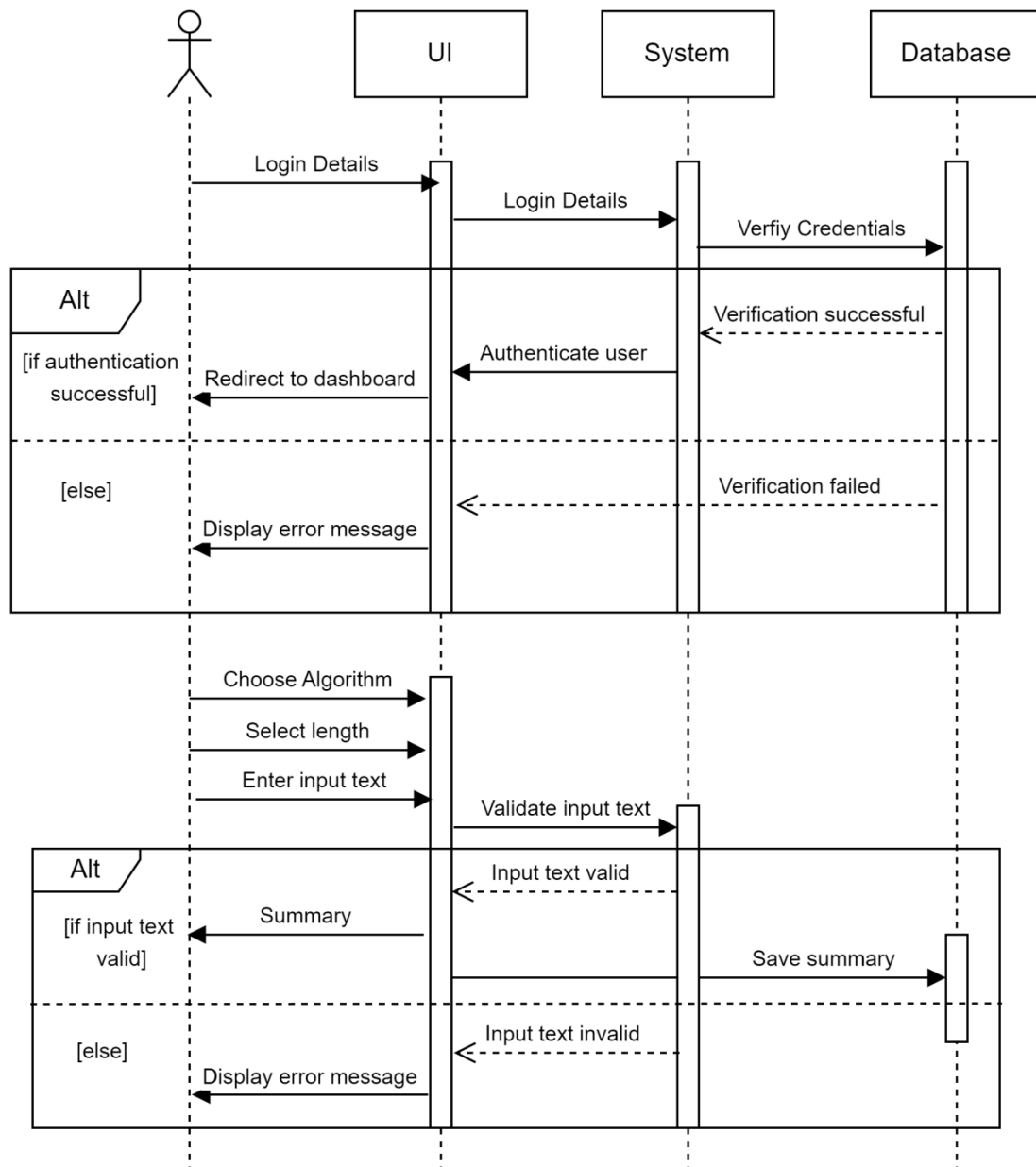


Figure 3.5: Sequence Diagram for Text summarization system

The process commences with the user providing login details via the UI, which are then forwarded to the system for verification against the database. An alternative scenario

arises if the verification is successful, where the system proceeds to authenticate the user, subsequently redirecting them to the dashboard. In case of verification failure, an error message is displayed to the user. Post-Authentication, the user can select an algorithm for text summarization, choose the length of the summary, and enter input text for summarization. The system validates the entered input text; if valid, a summary is displayed to the user and saved in the database, otherwise, an error message is shown.

3.1.3.4 Process modeling using Activity Diagram

Activity Diagrams facilitates the visual representation of workflows, activities, and decisions within a system. It aids in understanding the control flow of the system.

The flow of the application is illustrated by the activity diagram in Figure 3.5. Prior to utilizing the text summarization system, users are required to either login or register. Upon successful login, functionalities including summarization, viewing history, account deletion, history deletion, and logout become accessible. User input in the form of text and desired length is then utilized as input to the summarizer, enabling summarization using one of the provided algorithms.

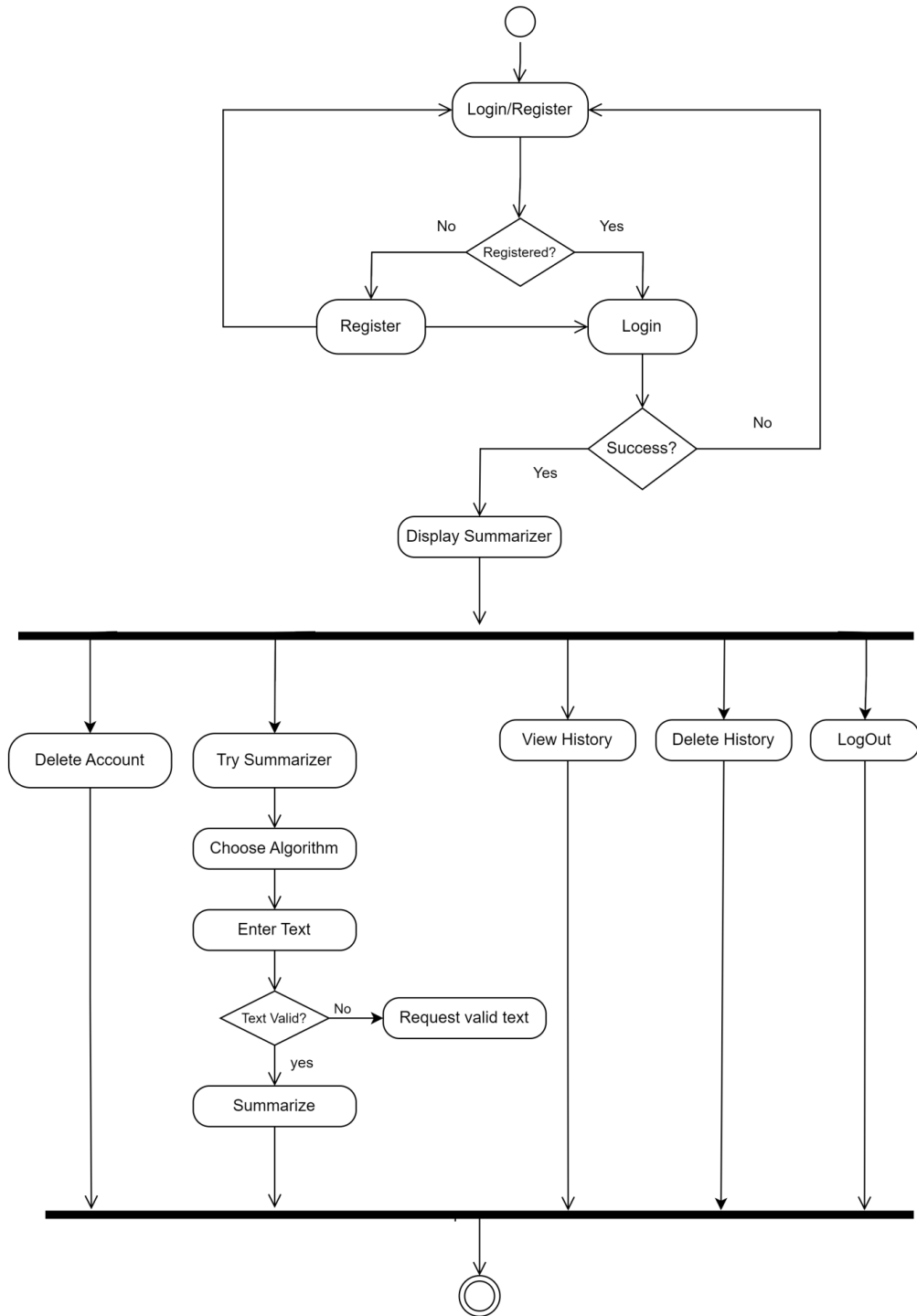


Figure 3.6: Activity Diagram for Text summarization system

CHAPTER 4: SYSTEM DESIGN

4.1 Design

System design involves creating a blueprint for the architecture, components, and interactions of a software system to meet specified requirements. It encompasses various aspects such as defining system modules, data flow, interfaces, and technologies.

i. System Design of Text Summarization system

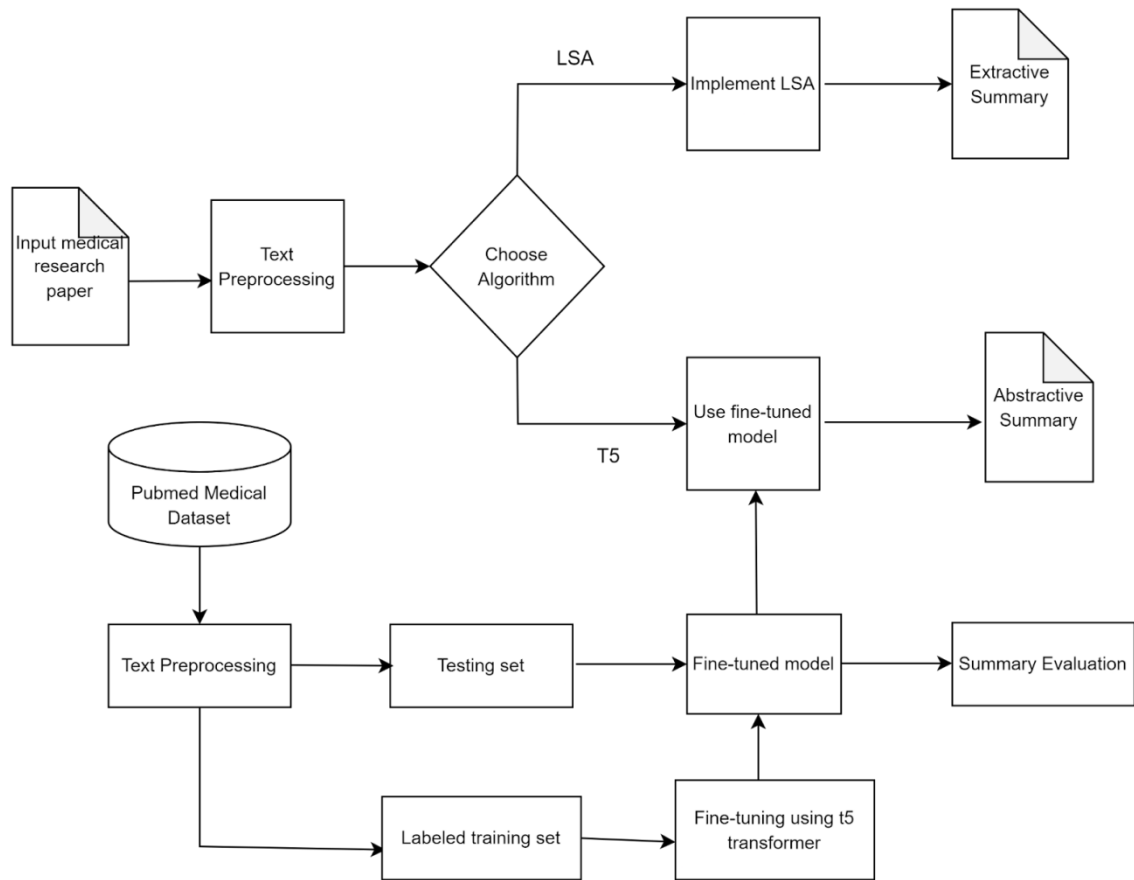


Figure 4.1: Block diagram of Text summarization system

The research paper text serves as the input to the text summarization system, undergoing text preprocessing before being processed according to the chosen algorithm—either LSA or a fine-tuned variant of the T5 model—for summarization. An extractive summary is produced by the LSA summarization, while an abstractive summary is generated by the

fine-tuned T5 model. Prior to its utilization, the fine-tuned T5 model is trained on the PubMed dataset.

4.1.1 Component Diagram

A component diagram is an UML diagram that showcases various components within a system and how they interact with each other. Components can represent software modules such as libraries, executables and other parts of a system. It is a visual representation that focuses on the static structure of a system.

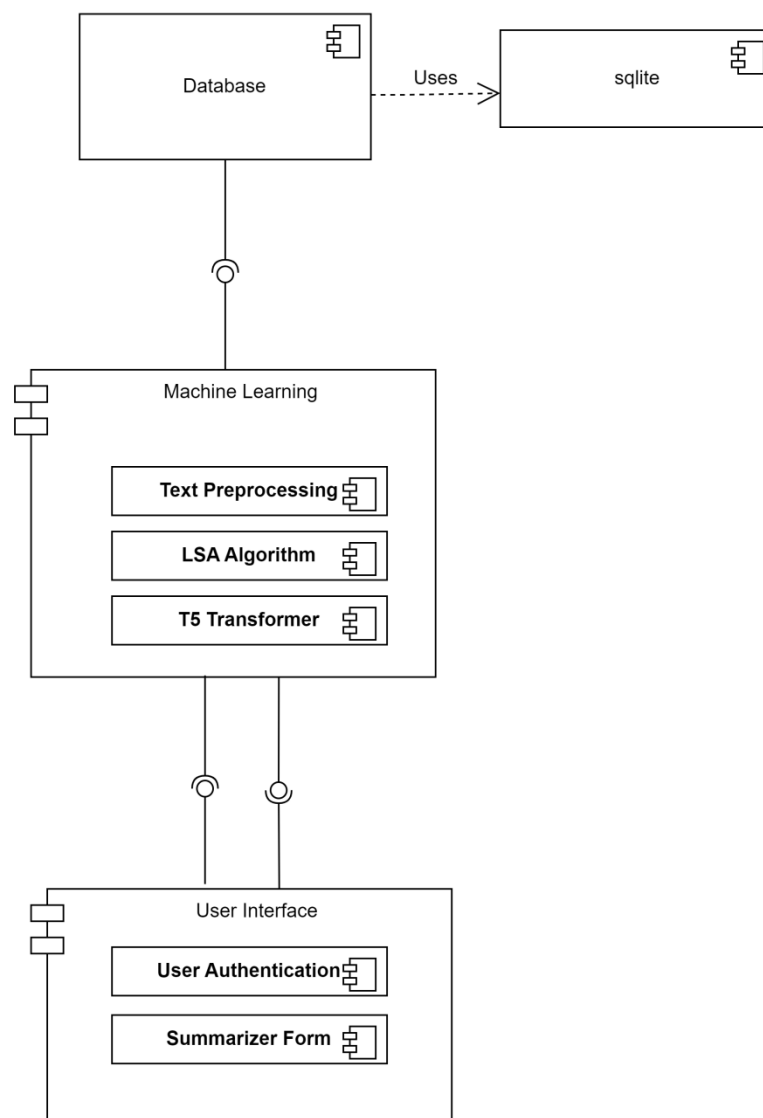


Figure 4.2: Component Diagram for Text summarization system

The component diagram illustrates the various components and their interactions within the text summarization system. The Database component serves as the backend for

storing and managing data. At the core of the system lies the Summarizer component, comprising three internal sub-components. The User Interface component facilitates user interaction with the system, encompassing two sub-components

4.1.2 Deployment Diagram

The deployment diagram describes the physical deployment of information generated by the software program on hardware components.

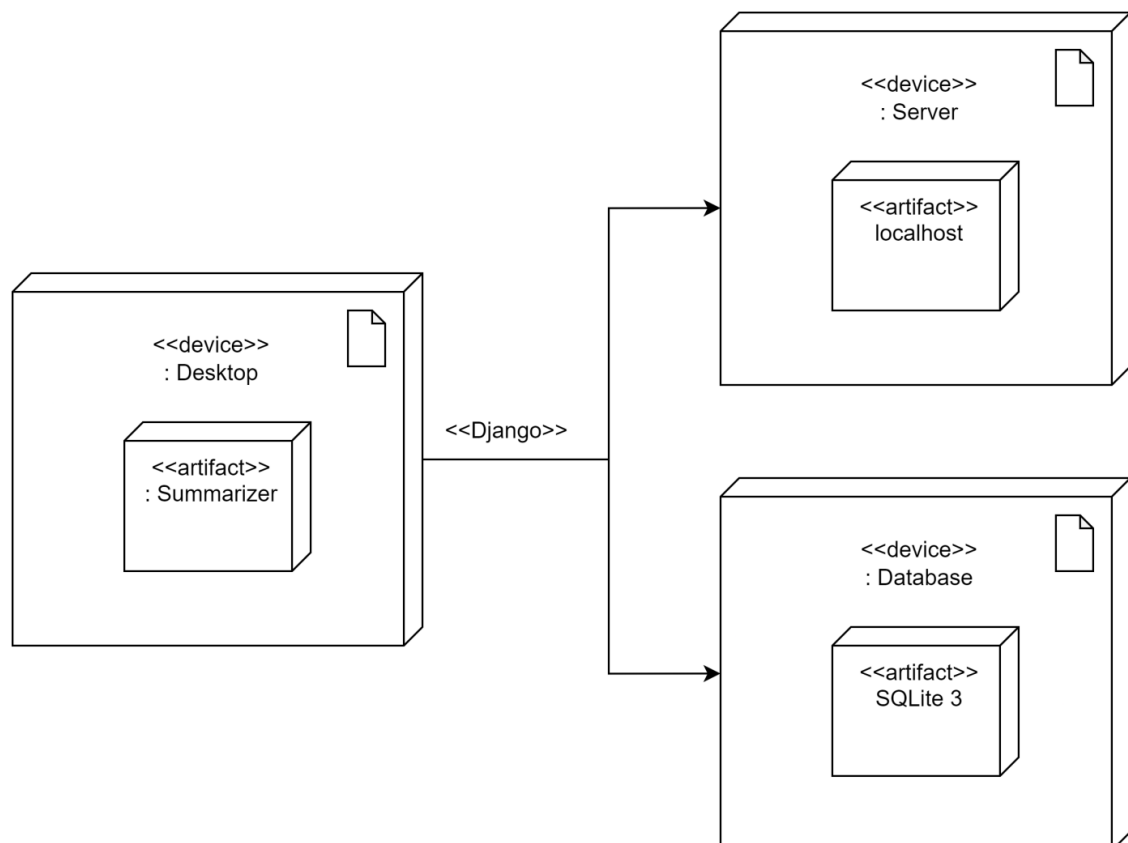


Figure 4.3: Deployment Diagram for Text summarization system

The deployment diagram illustrates how different components of the system are distributed across various devices. On the client-side, represented by the Desktop, users interact with the text summarization tool or application. The server, depicted as localhost, hosts the web application or service locally. The Database device houses an artifact labeled SQLite 3, indicating the use of an SQLite database for data storage. Both the server and the database are connected using Python-based web framework called Django.

4.2 Algorithm Details

4.2.1 Algorithm for LSA

Algorithm 1 LSA Summarization Algorithm with Step-by-Step Calculations

```

1: function LSASUMMARIZE(original_text, summary_length)
2:   Input: Original text (original_text), Desired summary length (summary_length)
3:   Output: Summarized text
4:                                     ▷ Text cleaning process
5:   clean_text ← CLEANTEXT(original_text)
6:   normalized_text ← NORMALIZETEXT(clean_text)
7:   sentences ← TOKENIZESENTENCES(normalized_text)
8:   tokenized_sentences ← TOKENIZEWORDS(sentences)
9:                                     ▷ SVD
10:  term_document_matrix ← CREATETERMDOCUMENTMATRIX(tokenized_sentences)
11:  dtm ← CALCULATEDTM(term_document_matrix)
12:  mean_centered_dtm ← MEANCENTER(dtm)
13:  covariance_matrix ← CALCULATECOVARIANCEMATRIXOPTIMIZED(mean_centered_dtm)
14:                                     ▷ Eigenvalue decomposition
15:  eigenvalues, eigenvectors ← PERFORMEIGENVALUEDECOMPOSITION(covariance_matrix)
16:                                     ▷ Dimensionality reduction
17:  U, S, Vt ← EXTRACTCOMPONENTS(eigenvectors, eigenvalues)
18:  selected_components ← CHOOSECOMPONENTS(eigenvalues, threshold)
19:  U_reduced, S_reduced, Vt_reduced ← REDUCEDDIMENSIONS(U, S, Vt, selected_components)
20:                                     ▷ Scoring sentences
21:  sentence_scores ← SCORESENTENCES(dtm, U_reduced, S_reduced)
22:                                     ▷ Select top sentences for summary
23:  summary_sentences ← SELECTTOPSENTENCES(sentence_scores, sentences, summary_length)
24:                                     ▷ Format and return the summary
25:  return FORMATSUMMARY(summary_sentences)
26: end function

```

Description

1. Term-Document Matrix (TDM):

Matrix $dtm \in \mathbb{R}^{(m \times n)}$

m = number of unique words

n = number of sentences

$dtm[i, j]$ = frequency of word i in sentence j

2. Mean Centering:

Matrix $mean_centered_dtm \in \mathbb{R}^{(m \times n)}$

$$X'_{ij} = X_{ij} - \bar{X}_j$$

Where, X'_{ij} is the mean centered value of the element in row i and column j ,

X_{ij} is the original value of the element in row i and column j ,

And $\bar{x}_j$ is the mean of column j .

3. Covariance Matrix:

Matrix covariance_matrix $\in \mathbb{R}^{(m \times n)}$

$$COV(X, Y) = \frac{1}{N} \sum_{i=0}^N (x_i - \bar{x})(y_i - \bar{y})$$

where,

-COV(X,Y) represents the covariance between X and Y,

-N is the number of sentences,

- x_i and y_i are the individual observations of variables X and Y

- and are the means (averages) of X and Y, respectively.

4. Eigen Decomposition

$$C = USU^T$$

where,

- C is the covariance matrix.

- U is the matrix containing the eigenvectors as its columns.

- S is a diagonal matrix containing the eigenvalues.

$$U = \begin{bmatrix} v_1 & v_2 & \dots & v_N \end{bmatrix}$$
$$S = \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_N \end{bmatrix}$$

5. Sentence Scoring:

For each sentence i , the score is computed as:

$$\text{Score}_i = \sum_{j=1}^N x_{ij} \cdot u_j \cdot s_{jj}$$

where:

- N is the number of principal components.
- x_{ij} represents the frequency of word j in sentence i in the document-term matrix.
- u_j is the j -th column of the matrix U containing eigenvectors.
- s_{jj} is the j -th diagonal element of the matrix S containing eigenvalues.

4.2.2 Algorithm for Text Summarization using T5 Transformer

Algorithm 1 Text Summarization with Fine-Tuned T5 Model

```

1: function T5SUMMARIZATION(dataset, hyperparameters)
2:   Input: Dataset of text-summary pairs (dataset), hyperparameters (hyperparameters)
3:   Output: Fine-tuned T5 model and generated summary
4:    $model \leftarrow \text{LoadT5Model}(\text{"t5-base"})$  ▷ Load base T5 model
5:    $train\_data, val\_data \leftarrow \text{SplitDataset}(\text{dataset}, 0.8)$  ▷ Split into 80/20 train/validation sets
6:    $tokenized\_train\_data \leftarrow \text{TokenizeData}(train\_data)$  ▷ Tokenize text and summaries
7:    $tokenized\_val\_data \leftarrow \text{TokenizeData}(val\_data)$ 
8:    $optimizer \leftarrow \text{SetOptimizer}(\text{hyperparameters})$  ▷ Set optimizer based on hyperparameters
9:    $batch \leftarrow \text{hyperparameters}[\text{"batch size"}]$ 
10:   $learning\_rate \leftarrow \text{hyperparameters}[\text{"lr"}]$ 
11:   $epochs \leftarrow \text{hyperparameters}[\text{"epochs"}]$ 
12:  for  $epoch \leftarrow 1$  to  $epochs$  do
13:    for  $batch \in tokenized\_train\_data$  do
14:       $loss \leftarrow \text{TrainModel}(model, batch, optimizer)$  ▷ Train on batch
15:       $\text{PrintLoss}(loss)$ 
16:    end for
17:    for  $batch \in tokenized\_val\_data$  do
18:       $val\_loss \leftarrow \text{ValidateModel}(model, batch)$  ▷ Validate on batch
19:       $\text{PrintValidationLoss}(val\_loss)$ 
20:    end for
21:  end for
22:   $\text{SaveModel}(model, \text{"finetuned\_t5\_model.pt"})$  ▷ Save fine-tuned model
23:   $input\_text \leftarrow \text{GetUserInput}()$  ▷ Get user input text for summarization
24:   $tokenized\_input \leftarrow \text{TokenizeText}(input\_text)$ 
25:   $summary\_tokens \leftarrow model.generate(tokenized\_input)$ 
26:   $summary \leftarrow \text{DecodeSummaryTokens}(summary\_tokens)$ 
27:  return  $model, summary$ 
28: end function

```

Description:

This algorithm harnesses the capabilities of fine-tuned T5 transformers to generate concise and informative summaries of long text documents.

1. Data Preprocessing

- **Dataset:** The algorithm operates on a dataset of text-summary pairs, specifically utilizing the PubMed dataset in this implementation. Each entry consists of an article and an abstract column which serves as the actual summary

- Splitting: The dataset is divided into training (80%) and validation (20%) sets to ensure robust model training and evaluation
- Tokenization: Both text and summaries undergo tokenization to prepare the data for input into the T5 transformer model.

2. Model Fine-Tuning

- Base Model: The algorithm leverages the pre-trained T5-base model, a powerful transformer architecture adapt at various text-based tasks.
- Hyperparameters: Fine-tuning is guided by specific hyperparameters, including batch size, learning rate, and number of training epochs.
- Training Loop: The model iteratively trains on batches of tokenized data, optimizing its parameters to generate summaries that align with the provided gold-standard summaries in the dataset.
- Validation: Validation loss is calculated during training to monitor model performance on unseen data and mitigate overfitting.

3. Summary Generation

- Tokenization: The input text is tokenized to match the model's expected input format.
- Summary Generation: The fine-tuned T5 model produces a sequence of summary tokens based on the input text.
- Decoding: The summary tokens are decoded back into natural language, resulting in a concise summary of the original text.

Hyper-parameters

Hyperparameters are key configuration settings that govern the training process and overall behavior of a machine learning model. They control aspects like the number of samples used in training and validation, the rate at which the model adjusts its weights during training, and the desired lengths of input text and generated summaries.

Table 4.1: HyperParameter

TRAIN_BATCH_SIZE	16	Number of samples used for training updates
VALID_BATCH_SIZE	8	Number of samples used for validation updates
TRAIN_EPOCHS	3	Number of times the entire training dataset is passed through the model
VAL_EPOCHS	1	Number of times the entire validation dataset is passed through the model
LEARNING_RATE	2e-5	Controls how quickly the model adjusts weights during training
SEED	42	Sets the random seed for reproducibility
MAX_LEN	256	Maximum input text length
SUMMARY_LEN	100	Desired length of generated summaries

A batch size of 16 for training strikes a balance between computational efficiency and model stability during training. Three training epochs were chosen to allow the model to converge to an optimal solution without overfitting or spending excessive time on training. A validation epoch after each training epoch was implemented to monitor the model's performance on unseen data. A small learning rate of 2e-5 was selected to prevent large parameter updates and subsequent model instability during training. The maximum input sequence length was set to 256 tokens, balancing efficient processing with the capacity to handle most documents. Finally, a summary length of 100 tokens was chosen to generate concise summaries capturing the document's core information.

4.2.3 Algorithm for Topic Extractor

Algorithm 1 Topic Extraction

```
1: function EXTRACTTOPICS( $D$ )
2:   Input: Text corpus  $D$  containing documents
3:   Output: Short topic sentence
4:   Initialize an empty dictionary word_counts to store word frequencies.
5:   Initialize document_count to 0 to keep track of the number of documents.
6:   for each document  $d$  in  $D$  do
7:     Preprocess text  $d$ : Convert to lowercase, remove special characters and digits.
8:     Tokenize text  $d$  into words.
9:     Tag words with parts of speech and filter out non-noun words.
10:    for each word  $w$  in  $d$  do
11:      if  $w$  exists in word_counts then
12:        Increment count of  $w$  in word_counts: word_counts[ $w$ ]  $\leftarrow$  word_counts[ $w$ ] + 1.
13:      else
14:        Add  $w$  to word_counts with count 1: word_counts[ $w$ ]  $\leftarrow$  1.
15:    Increment document_count: document_count  $\leftarrow$  document_count + 1.
16:  for each word  $w$  in word_counts do
17:    Calculate TF-IDF for  $w$ :
18:     $TF(w) = \frac{\text{word\_counts}[w]}{\sum \text{word\_counts.values}()}$ 
19:     $IDF(w) = \log\left(\frac{\text{document\_count}}{1 + \sum_{\text{count} > 0} 1}\right)$ 
20:     $TF\text{-}IDF(w) = TF(w) \times IDF(w)$ 
21:  Sort words by TF-IDF score in descending order.
22:  Extract top  $N$  words as topics.
23:  Generate short topic sentence by joining the top words.
24:  return Short topic sentence
```

4.2.4 Metrics used

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

ROUGE is a set of evaluation metrics used for assessing the quality of automatic summarization and machine translation outputs by comparing them to reference summaries. It measures the overlap and co-occurrence of n -grams (sequences of n words) and other linguistic units between the system-generated summaries and the human-generated reference summaries.

ROUGE primarily uses Recall, Precision, and F-score to quantify the quality of generated summaries.

- Recall measures the ability of the system to capture all the relevant information present in the reference summary.

$$\text{Recall} = \frac{\text{Overlapping number of } n\text{-grams}}{\text{Number of } n\text{-gram in the reference}}$$

- Precision assesses the accuracy of the generated summary by measuring the proportion of relevant information in it.

$$\text{Precision} = \frac{\text{Overlapping number of } n\text{-grams}}{\text{Number of } n\text{-gram in the prediction}}$$

- F-score combines both Recall and Precision into a single metric for a more comprehensive evaluation.

$$\text{F-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

ROUGE evaluates content overlap by comparing n-grams between the system-generated and reference summaries. It considers unigram, bigram, and trigram matches to capture both exact word overlap and more extended content similarity. Higher ROUGE scores indicate better agreement between system-generated and reference summaries. Researchers often report multiple ROUGE scores (e.g., ROUGE-1, ROUGE-2, ROUGE-L) to provide a comprehensive evaluation.

ROUGE-1 (Unigram Overlap):

ROUGE-1 evaluates the overlap of unigrams (single words) between the system-generated summary and the reference summary. It measures how many individual words in the system summary are also present in the reference summary.

ROUGE-2 (Bigram Overlap):

ROUGE-2 assesses the overlap of bigrams (two-word sequences) between the system-generated summary and the reference summary. It considers pairs of consecutive words.

ROUGE-L (Longest Common Subsequence):

ROUGE-L evaluates the overlap of the longest common subsequence of words between the system-generated summary and the reference summary. It considers word order and sentence structure.

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Implementation

5.1.1 Tools Used

i. Visual Studio Code

For our project, Visual Studio Code served as the primary code editor, chosen for its open-source nature, user-friendly interface, and versatile support for multiple programming languages. Its comprehensive feature set, encompassing debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, embedded Git, etc., made it an ideal choice for efficient and effective code development on the platform.

ii. Google Colaboratory

Google Colaboratory, often referred to as Colab, provides a Jupyter notebook environment hosted on Google Cloud servers, enabling collaborative and interactive coding. The project utilizes the free Graphics Processing Unit(GPU) resources for training purposes. Multiple users can collaborate in real-time on the same notebook. The tool also allows integration with Google Drive for easy storage and sharing.

iii. Draw.io

Draw.io is widely utilized in documentation for creating visually engaging diagrams that illustrate concepts, workflows, and system architectures. It enhances the clarity of information and aids in the effective communication of complex ideas. The tool provides shapes, icons and symbols for creating various types of diagrams including flowcharts, UML diagrams, network diagrams, mind maps, and entity-relationship diagrams.

iv. HTML, CSS and Javascript

HTML5 serves as the foundation of the front-end technology for our project, providing essential structure and presentation capabilities for content on the World Wide Web. Coupled with CSS3, which enhances the presentation layer of our documents authored in HTML5 by facilitating precise control over layout, fonts, and colors, it ensures a user-friendly interface. Additionally, JavaScript, known for its dynamic and multi-paradigm

nature, is utilized for both backend and frontend functionalities, contributing to the interactive and responsive features of our web application.

vii. Python and Django Framework

Django, a Python web framework, powers our back-end with efficient development tools and built-in features like ORM and authentication. It simplifies development while ensuring code reusability. Moreover, Django seamlessly integrates with SQLite, a lightweight database, for efficient data management within applications.

5.1.2 Implementation Details of LSA Algorithm

In this section, we present the implementation details of the LSA algorithm used for text summarization.

i. Text Cleaning

```
def clean_text(self, text):  
  
    """Clean the input text by removing special characters and digits."""  
  
    clean_chars = []  
  
    for char in text:  
  
        if char.isalnum() or char.isspace() or char in ",|?=":  
  
            clean_chars.append(char)  
  
    clean_text = "".join(clean_chars).strip()  
  
    clean_text = re.sub(r'\s+', ' ', clean_text)  
  
    return clean_text
```

The `clean_text` function removes special characters and digits from the input text, keeping only alphanumeric characters, spaces, and certain specified punctuation marks. It then removes any extra whitespace and returns the cleaned text.

ii. Sentence Tokenization

```
def tokenize_sentences(self, text):
```

```

"""Split text into sentences based on common sentence ending punctuation."""

sentences = []

current_sentence = ""

for char in text:

    if char in ".!?:":

        sentences.append(current_sentence)

        current_sentence = ""

    else:

        current_sentence += char

sentences.append(current_sentence)

return sentences

```

The `tokenize_sentences` function splits text into sentences. It iterates through each character, adding characters to a current sentence string until it encounters a common sentence ending punctuation (".", "!", "?"). Upon finding punctuation, it saves the current sentence, resets it to empty, and continues building the next sentence. This process ensures each sentence is separated based on proper punctuation.

iii. Term-Document Matrix Creation

```

def create_term_document_matrix(self, tokenized_sentences):

    term_document_matrix = {}

    for sentence_index, sentence in enumerate(tokenized_sentences):

        for word in sentence:

            if word not in term_document_matrix:

                term_document_matrix[word] = [0] * len(tokenized_sentences)

            term_document_matrix[word][sentence_index] += 1

```

```
return term_document_matrix
```

The `create_term_document_matrix` function constructs a term-document matrix (TDM) from tokenized sentences. It creates a dictionary where terms (words) are keys and values are lists representing word frequencies in each sentence. The function iterates through sentences and words. If a word is new (not encountered before), a list of zeros with the length of the total sentences is added to the dictionary for that word. Each time the word appears in a sentence, the count at the corresponding sentence index in the word's list is incremented. This process builds a comprehensive matrix showcasing word frequencies across all sentences.

iv. Singular Value Decomposition(SVD)

Singular Value Decomposition is applied to the mean-centered Document-Term Matrix (DTM). This mathematical technique helps identify the most important concepts in the text.

```
def perform_eigenvalue_decomposition(self, covariance_matrix):
```

```
    """Perform eigenvalue decomposition."""
```

```
    eigenvalues, eigenvectors = np.linalg.eig(covariance_matrix)
```

```
    sorted_indices = np.argsort(eigenvalues)[::-1]
```

```
    eigenvalues = eigenvalues[sorted_indices]
```

```
    eigenvectors = eigenvectors[:, sorted_indices]
```

```
    return eigenvalues, eigenvectors
```

The `perform_eigenvalue_decomposition` function conducts eigenvalue decomposition using NumPy's `np.linalg.eig` function, it computes the eigenvalues and corresponding eigenvectors of the covariance matrix. These results are then sorted in descending order based on the magnitude of the eigenvalues, achieved by sorting the eigenvalues and their corresponding eigenvectors accordingly.

v. Sentence Scoring and Summarization

The sentences are then scored based on their importance using the reduced singular values and vectors obtained from SVD. The algorithm selects the top-scoring sentences to generate a concise summary.

```
def score_sentences(self, dtm, U_reduced, S_reduced):  
  
    sentence_scores = []  
  
    for sentence_vector in dtm:  
  
        concept_scores = np.dot(sentence_vector, U_reduced)  
  
        weighted_scores = concept_scores * S_reduced  
  
        sentence_score = np.sum(weighted_scores)  
  
        sentence_scores.append(sentence_score)  
  
    return sentence_scores  
  
def select_top_sentences(self, sentence_scores, sentences, num_sentences=1):  
  
    top_indices = np.argsort(sentence_scores)[-num_sentences:][::-1]  
  
    selected_sentences = [sentences[i] for i in top_indices]  
  
    return selected_sentences
```

The first function, `score_sentences`, iterates through each sentence vector in the DTM. For each sentence, it calculates a score by multiplying the sentence vector with `U_reduced`, then element-wise multiplying the result with `S_reduced`, and finally summing the elements. The second function, `select_top_sentences`, sorts the scores in descending order and selects the top `num_sentences` corresponding indices. Finally, it retrieves the actual sentences from the original list using the selected indices and returns a list of the most important sentences based on their scores.

Figure 5.1: Working mechanism of LSA Algorithm

5.1.3 Implementation Details of T5 Algorithm

5.1.3.1. T5 Summarization

The T5 algorithm is implemented for text summarization using the pre-trained "t5-small" model from Hugging Face. The model is fine-tuned for summarization tasks and is well-suited for generating concise and coherent summaries.

```
def t5_summarize(original_text, summary_length):

    # Load pre-trained T5 model and tokenizer

    model = T5ForConditionalGeneration.from_pretrained("t5-small")

    tokenizer = T5Tokenizer.from_pretrained("t5-small")

    # Encode the input text and generate the summary

    inputs = tokenizer.encode("summarize: " + original_text, return_tensors="pt",
max_length=512, truncation=True)

    summary_ids = model.generate(inputs, max_length=summary_length,
length_penalty=2.0, num_beams=4, early_stopping=True)

    # Decode the generated summary

    summary = tokenizer.decode(summary_ids[0], skip_special_tokens=True)

    return summary
```

The `t5_summarize` function utilizes a pre-trained T5 model and tokenizer to generate a summary of the original text. First, the function loads the T5 model and tokenizer from the "t5-small" pre-trained model. It then encodes the input text using the tokenizer, prepending "summarize: " to the original text and ensuring a maximum length of 512 tokens. The encoded input is then passed to the T5 model's `generate` method, which generates a summary based on the input with specified parameters such as maximum length, length penalty, number of beams, and early stopping criteria. Finally, the generated summary is decoded using the tokenizer to obtain the human-readable text, which is returned by the function as the final summary.

5.1.3.2. T5 Fine-Tuned Model Summarization

```
class T5Summarizer:

    def __init__(self):

        self.model =
T5ForConditionalGeneration.from_pretrained("summarizer_app/models/pubmed_model"
)

        self.tokenizer = T5Tokenizer.from_pretrained("t5-base")

    def preprocess_text(self, text):

        clean_chars = []

        for char in text:

            if char.isalnum() or char.isspace() or char in ".,|?=":

                clean_chars.append(char)

        clean_text = "".join(clean_chars).strip()

        clean_text = re.sub(r'\s+', ' ', clean_text)

        return clean_text

    def format_summary(self, summary):

        summary = summary.strip()

        if summary.startswith(":"):

            summary = summary[1:].strip()

        summary = summary.capitalize()

        if not summary.endswith('.'):

            summary += '.'

        return summary
```

```

def summarize(self, original_text, summary_length):

    original_text = self.preprocess_text(original_text)

    inputs = self.tokenizer.encode("summarize: " + original_text, return_tensors="pt",
max_length=512, truncation=True)

    summary_ids = self.model.generate(inputs, max_length=summary_length,
length_penalty=2.0, num_beams=4, early_stopping=True)

    summary = self.tokenizer.decode(summary_ids[0], skip_special_tokens=True)

    formatted_summary = self.format_summary(summary)

    return formatted_summary

```

The T5Summarizer class is designed to perform text summarization using the fine-tuned T5 transformer model. It initializes with a T5 transformer model for conditional generation and a tokenizer from the Hugging Face transformers library. The preprocess_text method is used to clean and preprocess the input text before summarization. The format_summary method adjusts the format of the generated summary. The summarize method encodes the preprocessed text with the special token "summarize:" using the tokenizer. The encoded input is then passed to the model's generate method to generate the summary. The generated summary is decoded using the tokenizer, removing special tokens. Finally, the formatted summary is returned.

i. Block Diagram of Fine-tuning (T5 Transformer)

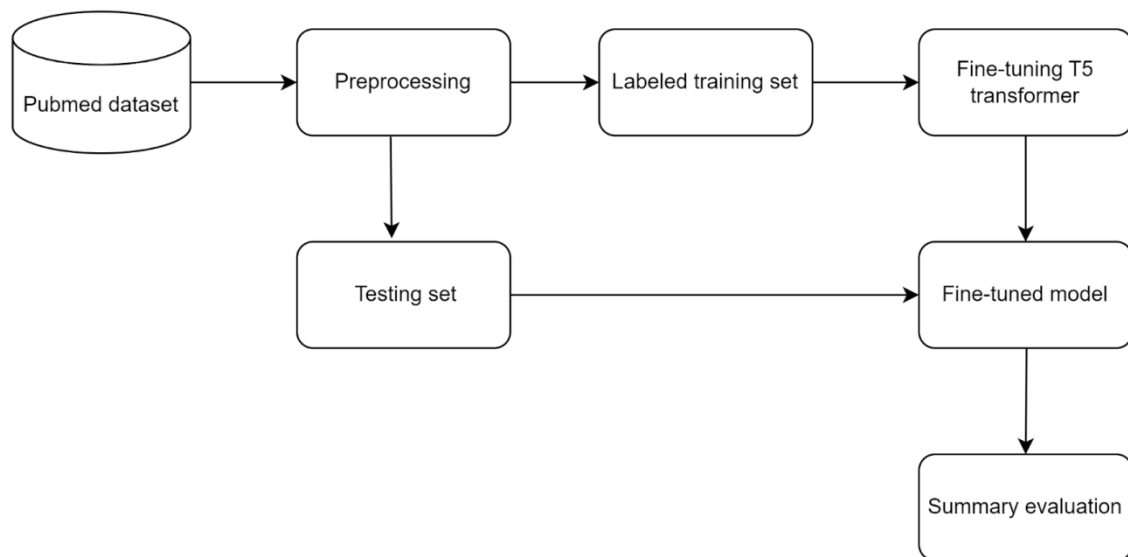


Figure 5.2: Block diagram for Fine-tuning using T5 transformer

In fine-tuning, the process begins with preprocessing the data which is the Pubmed medical article dataset, the T5 base model is selected for its text generation capabilities. Tokenization is applied to both input text and target summaries, followed by configuring fine-tuning parameters such as batch size and learning rate. The model is then fine-tuned on the dataset, adjusting its weights to generate accurate summaries. Evaluation metrics like ROUGE scores assess the quality of the generated summaries against actual summaries. Finally, the fine-tuned model is deployed to process unseen data, extending its application beyond the training dataset.

5.1.3 Implementation Details of Topic Extraction

```
class TopicExtractor:
```

```
    def __init__(self):
```

```
        self.word_counts = {}
```

```
        self.document_count = 0
```

```
    def preprocess_text(self, text):
```

```

text = text.lower()

# Remove special characters and digits

text = re.sub(r'^a-zA-Z\s', '', text)

return text

def update_word_counts(self, words):

    # Update word counts

    for word in words:

        if word in self.word_counts:

            self.word_counts[word] += 1

        else:

            self.word_counts[word] = 1

    self.document_count += 1

def calculate_tfidf(self, word):

    # Calculate TF-IDF for a word

    tf = self.word_counts[word] / sum(self.word_counts.values())

    idf = math.log(self.document_count / (1 + sum(1 for count in
self.word_counts.values() if count > 0)))

    return tf * idf

def extract_topics(self, text):

    # Preprocess text

    text = self.preprocess_text(text)

    # Tokenize text into words

    words = word_tokenize(text)

```

```

# Tag words with parts of speech
tagged_words = pos_tag(words)

# Filter out non-noun words
noun_words = [word for word, pos in tagged_words if pos.startswith('N')]

# Update word counts
self.update_word_counts(noun_words)

# Calculate TF-IDF for each word
tfidf_scores = {word: self.calculate_tfidf(word) for word in noun_words}

# Sort words by TF-IDF score in descending order
sorted_words = sorted(tfidf_scores.items(), key=lambda x: x[1], reverse=True)

# Extract top N words as topics
num_topics = min(5, len(sorted_words)) # Extract top 5 topics
topics = [word for word, _ in sorted_words[:num_topics]]

# Generate short topic sentence
short_topic = " ".join(topics) + "."

return short_topic

```

The topic extraction class automatically identifies key themes within a text document. During pre-processing, the text is converted to lowercase and undergoes special character and digit removal. Subsequently, noun words are extracted through parts-of-speech tagging, and word counts are updated internally. To pinpoint significant topics, the `extract_topics` method calculates TF-IDF for each noun word. TF-IDF considers both the frequency of a word's appearance in the document and its rarity across documents in the dataset. Words assigned high TF-IDF scores are viewed as more indicative of the document's content. Finally, the top N words are extracted based on their TF-IDF scores and combined into a concise topic sentence, summarizing the document's central themes.

5.2. Testing

Testing is an integral part of the software development life cycle, serving as a critical mechanism for ensuring the reliability, functionality, and overall quality of the system. This section delineates the test cases devised for both unit testing and system testing.

5.2.1. Test Cases for Unit Testing

Unit Testing is a vital process within software development, involving the isolated examination of individual code units. In our system unit testing was performed to test the individual component of the system as given below:

Table 5.1: User Registration Text Case

Test Case	Description	Test Data	Expected Result	Actual Result	Result
Registration Test	User enters invalid email.	Email: abc Username: suman Password: Password@1	This is not a valid email format.	As expected	Pass
Registration Test	User enters password without required characters.	E-mail: sumankhadka16@gmail.com Username: suman@1 Password: password	Password must contain at least 8 characters, including 1 each number, uppercase and special character.	As expected	Pass
Registration Test	User successfully registered.	Email: suman@gmail.com Username: suman@1 Password: Password@1	Account created successfully	As expected	Pass

This table outlines various test cases designed to validate the user registration functionality. It covers scenarios with invalid email formats, insufficient password

complexity, and successful registration. All test cases passed successfully, indicating that the registration process functions as intended.

Table 5.2: User Login Test Case

Test Case	Description	Test Data	Expected Result	Actual Result	Result
Login Test	User enters wrong Username	Username: saman	Wrong Username or password	As expected	Pass
Login Test	User enters wrong password	Username: suman@1 Password: pssword@1	Wrong Username or password	As expected	Pass
Login Test	User enters correct username and password	Username: suman@1 Password: Password@1	Login successful	As expected	Pass

This table outlines the test cases designed to validate the user login functionality. It covers scenarios with incorrect usernames, incorrect passwords, and successful login. All test cases passed successfully, indicating that the login process functions as intended.

Table 5.3: Input Text Validation Test Case

Test Case	Description	Test Data	Expected Result	Actual Result	Result
Input Text Validation Test	User does not enter any input text.	Empty	Please enter some data before proceeding.	As expected	Pass
Input Text Validation Test	User enters input text with character count less than 30.	“Physical and Spiritual activity.”	Please enter a text with at least 30 characters.	As expected	Pass

This table summarizes the test cases designed to validate user input text. It covers scenarios where the user enters no text and text with a character count below the minimum threshold. All test cases passed successfully, ensuring the input validation for text length works as expected.

5.2.2. Test Cases for System Testing

System testing is a comprehensive evaluation of the entire system, focusing on interactions, integrations, and overall functionality. The following test cases are designed to ensure the holistic functionality of the system.

Table 5.4: System Test Case

Test Case	Description	Input Test Data	Expected Result	Actual Result	Remarks
Algorithm Selection Test	Summarize according to selected algorithm.	User selects algorithm LSA	System successfully switches to algorithm LSA	As expected	Pass
Summary Length Generation Test	System generates different length of summary	Summary length: 100 characters, Text:[input text]	System generates a summary of maximum 100 characters from the given input text	As expected	Pass
Page Routing Accuracy Test	Successfully routing for displaying correct pages	User navigates to each section [Homepage, About us, Summarizer]	System displays the correct pages corresponding to each section.	As expected	Pass
User Authentication Test	Successful login and logout with user	User logs in, user's History: [history data]	System displays user information and history upon	As expected	Pass

	information		successful login and logout		
--	-------------	--	--------------------------------	--	--

This table summarizes test cases designed to validate features of the application. It covers functionalities like algorithm selection, summary length generation, page routing, user authentication, and potentially historical data access. All test cases passed successfully, indicating that these features work as intended.

5.3. Result Analysis

Result analysis is a critical phase in the testing process, providing insights for informed decision-making. In the case of a T5 model fine-tuned on medical research papers, a comprehensive analysis was conducted to assess the performance across multiple dimensions, including ROUGE scores, summary length distribution, and training loss.

The following section presents the summaries generated by two distinct algorithms, Latent Semantic Analysis (LSA) and T5, applied to a sample input text. Alongside these summaries, their respective ROUGE scores have been calculated against a reference summary to evaluate their performance in capturing the essence of the original text.

The screenshot displays a web application for text summarization. The main content area is titled 'Text Summarization' and is divided into two primary sections: 'Original Text' and 'Summarized Text'. The 'Original Text' section shows a long paragraph of text and a 'Word Count: 4963'. Below this, there is a 'Choose Algorithm:' dropdown menu currently set to 'LSA', and a 'Summary Length: 7 Sentences' option. A blue 'Summarize' button is located at the bottom of this section. The 'Summarized Text' section displays a shorter version of the text, with 'Word Count: 274 | Sentence Count: 7'. A 'Open Dashboard' button is visible at the bottom of this section. The top navigation bar includes links for 'Homepage', 'About Us', 'Try Our Summarizer', 'Slides', and 'Logout'.

Figure 5.3: Summary generated by LSA for sample text

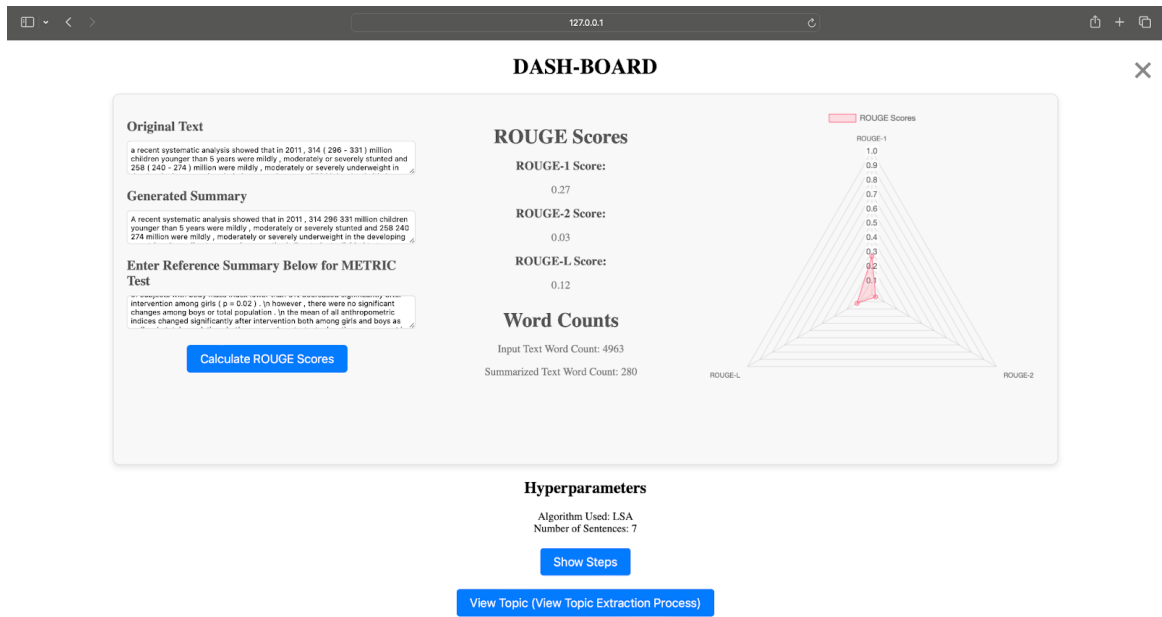


Figure 5.4: Rouge score for the summary generated by LSA for sample text

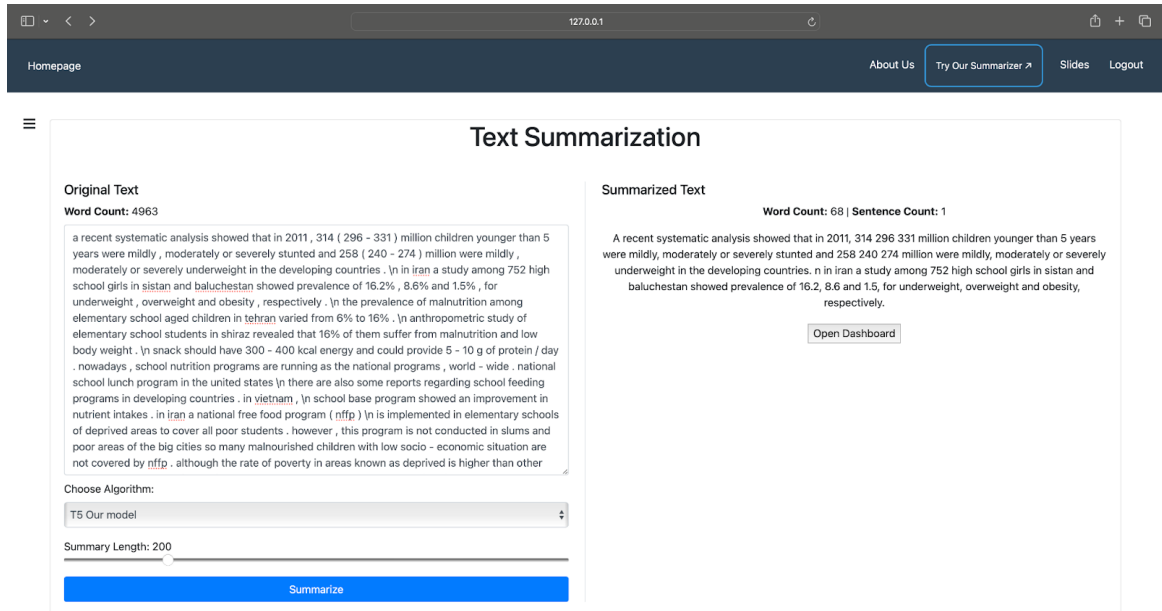


Figure 5.5: Summary generated by fine-tuned T5 model for sample text

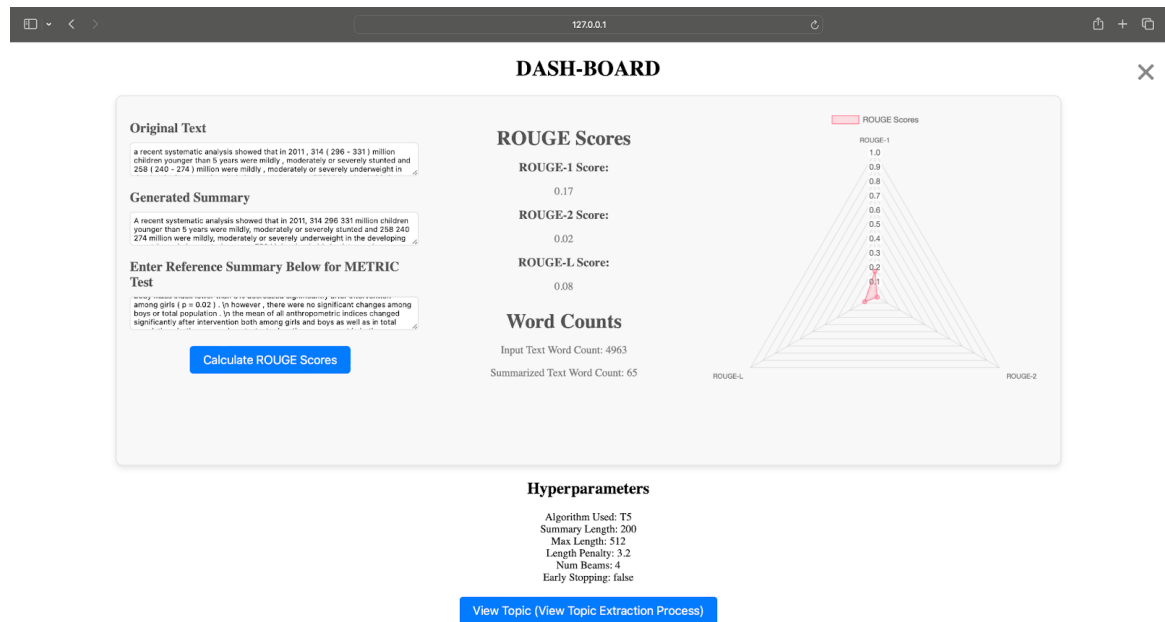


Figure 5.6: Rouge score for the summary generated by T5 for sample text

The summary generated by our T5 model provides a condensed representation of the original text while LSA provides extractive summary. The ROUGE scores suggest moderate alignment between the two summaries, further refinement of the model or adjustments to the hyperparameters may enhance its summarization capabilities.

Table 5.5: Overall Rouge Scores for LSA and T5

Dataset	Evaluation Metric	Algorithm	Recall	Precision	F-measure
PubMed Article Summarization Dataset	Rouge-1	LSA	0.57494	0.91547	0.68250
		T5	0.37138	0.33803	0.35020
	Rouge-2	LSA	0.46807	0.83297	0.57216
		T5	0.15098	0.13489	0.14095
	Rouge-L	LSA	0.57490	0.91537	0.68244
		T5	0.33772	0.30724	0.31839

This table presents a comparison of two summarization algorithms (LSA and T5) evaluated on the PubMed Article Summarization Dataset using three Rouge metrics (Rouge-1, Rouge-2, Rouge-L). LSA consistently outperforms T5 across all three Rouge metrics, indicating that it generally generates summaries with higher quality and relevance compared to T5.

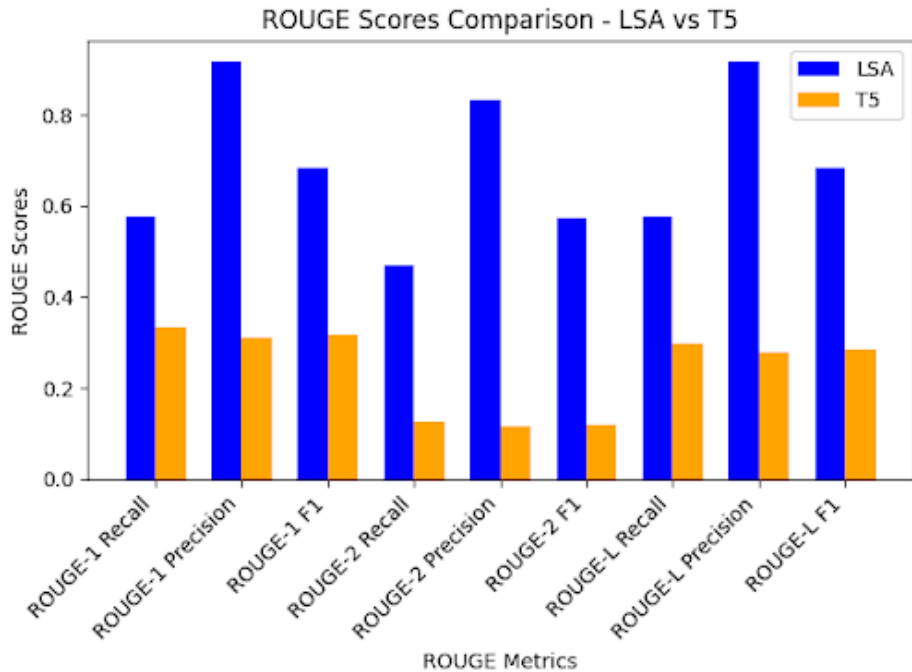


Figure 5.7: Rouge Score Comparison of LSA and T5

From the graph in figure 5.9, it is clear that LSA outperforms T5 on medical paper summaries due to its extractive approach: higher ROUGE-L reflects better capture of specific terms & phrases crucial for factual accuracy in the medical domain, even if summaries lack novelty. T5 might be suitable for conciseness & diverse perspectives, but requires careful evaluation for potential factual errors.

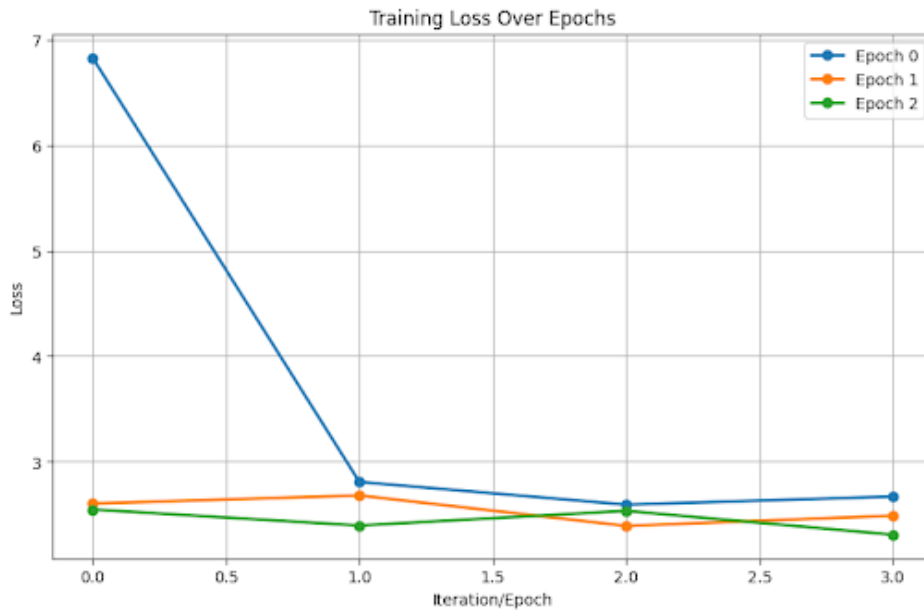


Figure 5.8: Training Loss over Epochs

A consistent decrease in the loss function was observed as the number of epochs increased during the training process. The training loss is a measure of how well the model is performing on the training data. A lower training loss indicates that the model is learning better. Three epochs were taken for training the model to strike a balance between training time and ensuring sufficient learning for the model to generalize well to unseen data. The graph shows that the training loss decreases over time, which means that the model is learning.

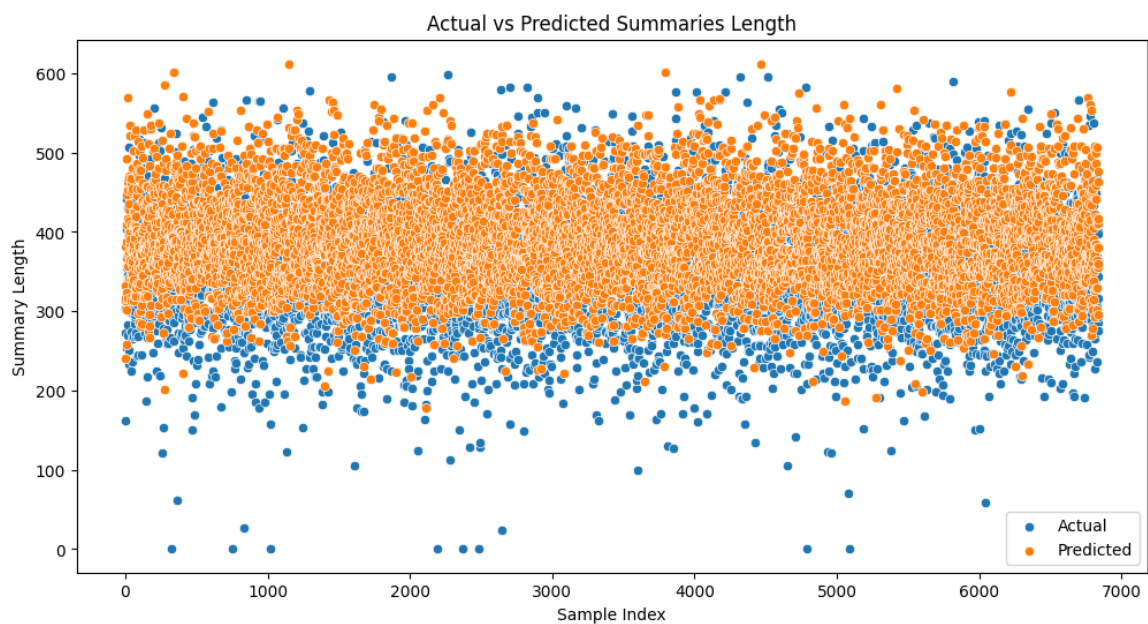


Figure 5.9: Comparison between actual and predicted summary length

The scatter plot in figure 5.5 offers a comparative analysis between the lengths of the generated summaries and those of the actual summaries in the dataset. It reveals that, on the whole, the generated summaries closely align with the lengths of the actual summaries, particularly when the target summary length is approximately set around 400.

CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATION

6.1 Conclusion

This project aimed to address the challenge of summarizing medical research papers by developing a system that takes into account the domain-specific terminology. We implemented two text summarization methods: Latent Semantic Analysis (LSA) and Text Summarization using T5 Transformer. Both methods were successful in generating concise and informative summaries of long text documents.

The LSA method involved creating a term-document matrix, mean centering, calculating the covariance matrix, performing singular value decomposition, reducing dimensionality, scoring sentences, and generating a summary. The T5 Transformer method involved data preprocessing, model fine-tuning, and summary generation.

We evaluated the quality of the generated summaries using the ROUGE metrics, which measure the overlap and co-occurrence of n-grams between the system-generated summaries and the human-generated reference summaries. The results showed that our system was able to generate summaries that align well with the provided gold-standard summaries in the dataset.

6.2 Future Recommendation

While the current system has shown promising results, there are several areas where future work could improve upon.

1. **Advanced Transformer Models:** Exploring more advanced transformer models such as BART or GPT-3 for abstractive summarization could potentially improve the quality of the generated summaries.
2. **Semantic Similarity and Coherence Measures:** Incorporating semantic similarity and coherence measures in the evaluation metrics could provide a more comprehensive assessment of the quality of the summaries.
3. **Different Domains and Languages:** Applying the system to different domains and languages could make it more versatile and useful for a wider range of users.

- 4. User-Friendly Interface:** Developing a user-friendly interface and deploying the system as a web or mobile application could make it more accessible to researchers and practitioners.

By continuing to refine and expand upon this work, we can further enhance the capabilities of automatic text summarization systems, particularly in the biomedical domain. This will ultimately aid researchers, clinicians, and healthcare professionals in swiftly accessing relevant information amidst the overwhelming volume of data available on biomedical databases.

REFERENCES

- [1] M. Allahyari, S. Pouriyeh, M. Assefi, S. Safaei, E. D. Trippe, J. B. Gutierrez, and K. Kochut, "Text summarization techniques: a brief survey," arXiv preprint arXiv:1707.02268, 2017.
- [2] Fraser, N., Brierley, L., Dey, G., Polka, J. K., Pálffy, M., Nanni, F., & Coates, J. A. (2021). "The evolving role of preprints in the dissemination of COVID-19 research and their impact on the science communication landscape." *PLoS Biology*, 19(4), e3000959. <https://doi.org/10.1371/journal.pbio.3000959>
- [3] R. Mihalcea and P. Tarau, "TextRank: Bringing Order into Text," in *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, Barcelona, Spain, Jul. 2004, pp. 404-411.
- [4] J. Pilault, R. Li, S. Subramanian, and C. Pal, "On Extractive and Abstractive Neural Document Summarization with Transformer Language Models," 2020. [Online]. Available: <https://arxiv.org/pdf/1909.03186.pdf>
- [5] Ramya, T. E., & Maghesh, N. (2020). "Text summarization using latent semantic analysis," *International Journal of Scientific Research in Computer Science and Engineering*, 8(1), 87-92. Available: <https://www.isroset.org/>
- [6] Vaswani, N. M. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," arXiv preprint arXiv:1706.03762, 2017.
- [7] Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," arXiv preprint arXiv:1910.10683, 2019.
- [8] T. Landauer, P. Foltz, and D. Laham, "An Introduction to Latent Semantic Analysis," *Discourse Processes*, vol. 25, pp. 259-284, 1998, doi: 10.1080/01638539809545028

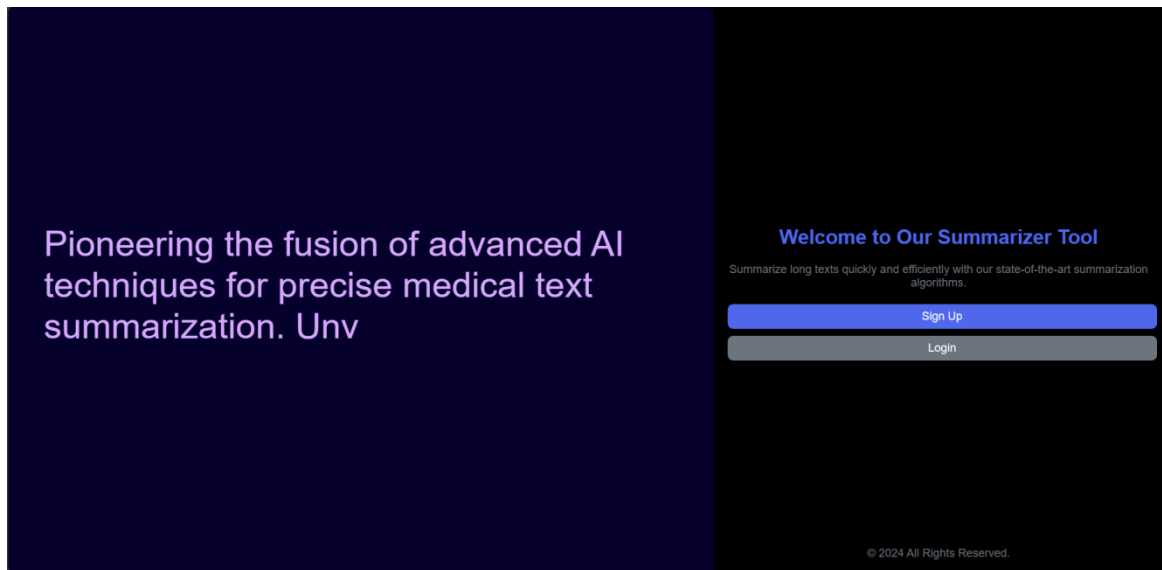
- [9] HealthCare IT Today. "More Healthcare Data Means Growth for Switch" Available: <https://www.healthcareittoday.com/>
- [10] H.P. Luhn, "The automatic creation of literature abstracts," Available: <https://courses.ischool.berkeley.edu/i256/f06/papers/luhn58.pdf>
- [11] Y. J. Kumar, O. S. Goh, H. Basiron, N. H. Choon, and P. C. Suppiah, "A Review on Automatic Text Summarization Approaches," Journal of Computer Science, Faculty of Information and Communication Technology, Universiti Teknikal Malaysia Melaka, Melaka, Malaysia, Available: <https://thescipub.com/pdf/jcssp.2016.178.190.pdf>
- [12] R. Jain, A. Jangra, S. Saha, and A. Jatowt, "A Survey on Medical Document Summarization," J. ACM, vol. 37, no. 4, Article 111, Aug. 2018, pp. 29, doi: 10.1145/1122445.1122456.
- [13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.
- [14] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," Available: <https://aclanthology.org/N19-1423>
- [15] Mahipal Nehra, "6 Stages of the Agile Development Lifecycle," Available: <https://www.decipherzone.com/blog-detail/agile-development-lifecycle>
- [16] F. Kiyomarsi, "Evaluation of Automatic Text Summarizations based on Human Summaries," Procedia - Social and Behavioral Sciences, vol. 192, pp. 83-91, 2015. doi: 10.1016/j.sbspro.2015.06.013.
- [17] D. Yadav, J. Desai, and A. K. Yadav, "Automatic Text Summarization Methods: A Comprehensive Review," 2022. Available: <https://aclanthology.org/N19-1423>
- [18] R. Nallapati, F. Zhai, and B. Zhou, "SummaRuNNer: A Recurrent Neural Network Based Sequence Model for Extractive Summarization of Documents,"

in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 31, 2016, pp. 10958. doi: 10.1609/aaai.v31i1.10958.

- [19] J. Hake, M. Crowley, A. Coy, D. Shanks, A. Eoff, K. Kirmer-Voss, G. Dhanda, and D. J. Parente, "Quality, Accuracy, and Bias in ChatGPT-Based Summarization of Medical Abstracts," *Ann Fam Med*, vol. 22, no. 2, pp. 113-120, Mar-Apr 2024. doi: 10.1370/afm.3075. PMID: 38527823.
- [20] Pubmed Summarization Dataset. Available: <https://huggingface.co/datasets/ccdv/pubmed-summarization>

APPENDICES

SNAPSHOTS



Register

First name:

Last name:

Username:

Required. 150 characters or fewer.
Letters, digits and @./+/_ only.

Email address:

Password:

Confirm Password:

Register

Already have an account? [Login](#)

Login

Username

Password

Login

Don't have an account? [Sign up](#)

[Back to Main](#)

Homepage

About Us

Try Our Summarizer ↗

Slides

Logout

Text Summarization

Original Text

Word Count: 0

It's The World Soil Day is being observed today globally as well as in Nepal with the objective of raising public awareness on the significance of healthy soil and for the sustainable management of the soil fertility.The United Nations General Assembly had in December 2013 declared December 5, 2014 as the World Soil Day and it was formally marked throughout the world since then. Nepal has been observing the Day since 2015. The theme of the World Soil Day this year is 'Soils: where food begins.' Soil is at the heart of all agricultural activities, food security, nutrition security and climate conservation. The World Soil Day programme reiterates the importance of soil for mankind and the crucial need for its conservation and proper management while at the same time increasing its fertility, the Department of Agriculture said.Director General of the Department, Dr Rewati Raman Poudel said that the debate about food and nutritional security, sustainable agriculture development, conservation of bio-diversity and organic agriculture will have no meaning without the conservation, promotion and proper management of soil. The soil fertility is deteriorating

Choose Algorithm:

LSA

Summary Length: 2

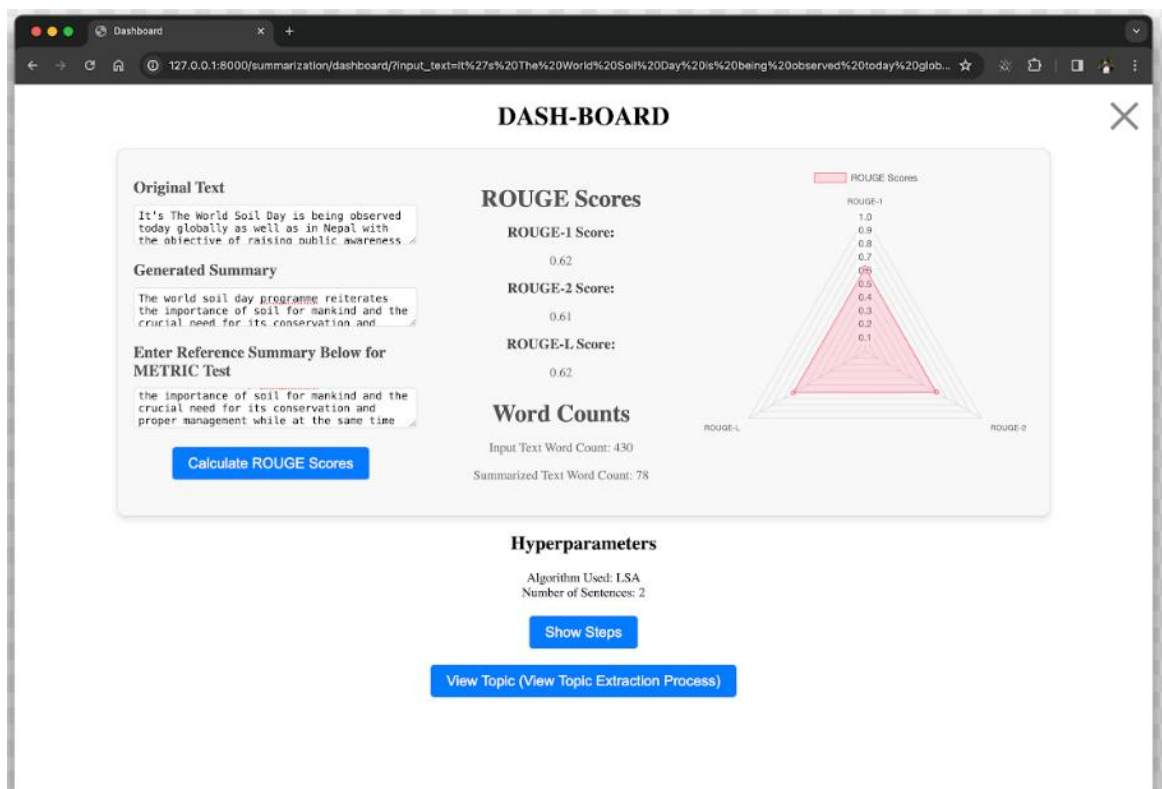
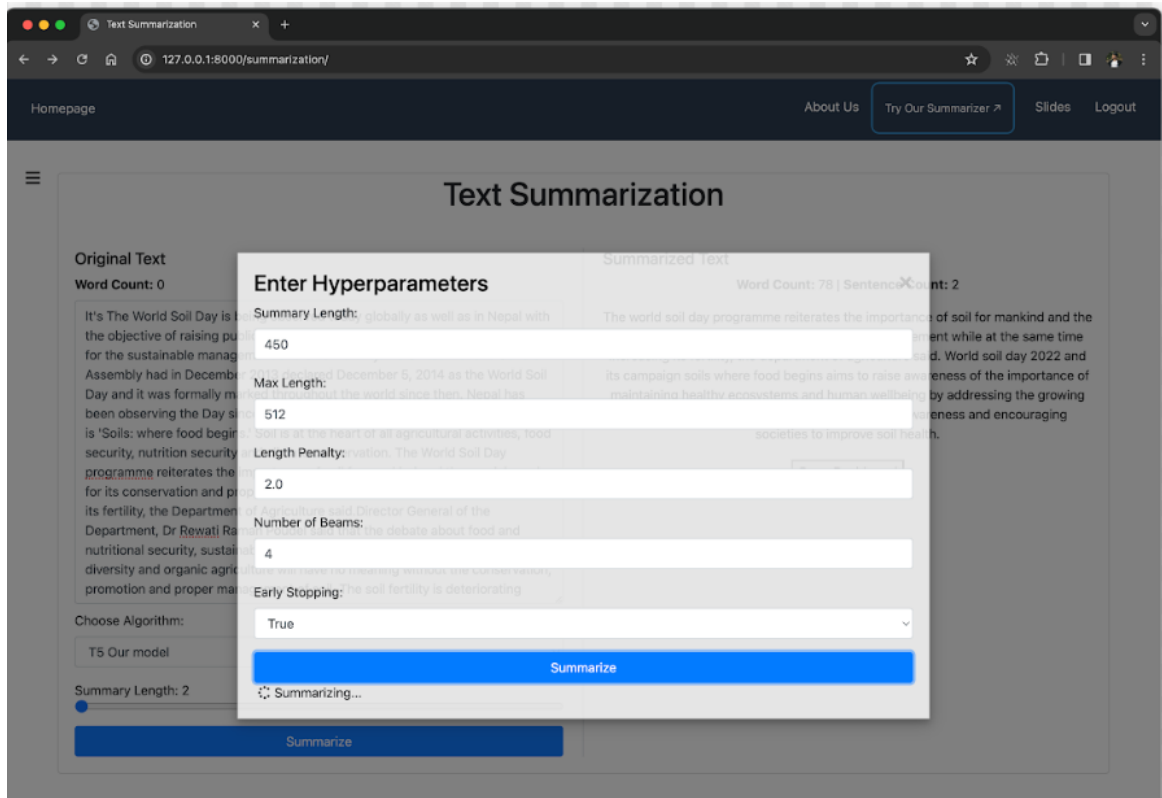
Summarize

Summarized Text

Word Count: 78 | Sentence Count: 2

The world soil day programme reiterates the importance of soil for mankind and the crucial need for its conservation and proper management while at the same time increasing its fertility, the department of agriculture said. World soil day 2022 and its campaign soils where food begins aims to raise awareness of the importance of maintaining healthy ecosystems and human wellbeing by addressing the growing challenges in soil management, increasing soil awareness and encouraging societies to improve soil health.

Open Dashboard





Hyperparameters

Algorithm Used: LSA
Number of Sentences: 2

Show Steps

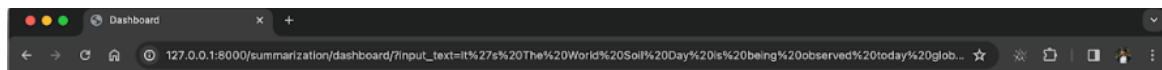
Step 1: input text

It's The World Soil Day is being observed today globally as well as in Nepal with the objective of raising public awareness on the significance of healthy soil and for the sustainable management of the soil fertility. The United Nations General Assembly had in December 2013 declared December 5, 2014 as the World Soil Day and it was formally marked throughout the world since then. Nepal has been observing the Day since 2015. The theme of the World Soil Day this year is 'Soils: where food begins.' Soil is at the heart of all agricultural activities, food security, nutrition security and climate conservation. The World Soil Day programme reiterates the importance of soil for mankind and the crucial need for its conservation and proper management while at the same time increasing its fertility, the Department of Agriculture said. Director General of the Department, Dr Rewati Raman Poudel said that the debate about food and nutritional security, sustainable agriculture development, conservation of bio-diversity and organic agriculture will have no meaning without the conservation, promotion and proper management of soil. The soil fertility is deteriorating throughout the world including in Nepal in the recent years with the declining physical, chemical and biological features of the soil. Therefore, this problem of declining soil fertility has been taken as the common global problem. Poudel said the World Soil Day is being marked with the main goal of raising extensive public awareness to tackle this growing problem of loss in soil fertility. The World Soil Day is being celebrated at the national level today in Nepal amidst various programmes under the aegis of the Department, Central Agricultural Laboratory, the National Soil Science Research Centre (NARS), Food and Nutrition Security Improvement Project, Rural Enterprises and Economic Development Project, United Nations, Food and Agriculture Organization and the Nepalese Society of Soil Science. The UN has said that over the last 70 years, the level of vitamins and nutrients in food has drastically decreased, and it is estimated that 2 billion people worldwide suffer from lack of micronutrients, known as hidden hunger because it is difficult to detect. Soil degradation induces some soils to be nutrient depleted losing their capacity to support crops, while others have such a high nutrient concentration that represent a toxic environment to plants and animals, pollutes the environment and cause climate change. World Soil Day 2022 and its campaign 'Soils: Where food begins' aims to raise awareness of the importance of maintaining healthy ecosystems and human well-being by addressing the growing challenges in soil management, increasing soil awareness and encouraging societies to improve soil health.

Step 2: clean text

Its The World Soil Day is being observed today globally as well as in Nepal with the objective of raising public awareness on the significance of healthy soil and for the sustainable management of the soil fertility. The United Nations General Assembly had in December 2013 declared December 5, 2014 as the World Soil Day and it was formally marked throughout the world since then. Nepal has been observing the Day since 2015. The theme of the World Soil Day this year is Soils where food begins. Soil is at the heart of all agricultural activities, food security, nutrition security and climate conservation. The World Soil Day programme reiterates the importance of soil for mankind and the crucial need for its conservation and proper management while at the same time increasing its fertility, the Department of Agriculture said. Director General of the Department, Dr Rewati Raman Poudel said that the debate about food and nutritional security, sustainable agriculture development, conservation of biodiversity and organic agriculture will have no meaning without the conservation, promotion and proper management of soil. The soil fertility is deteriorating throughout the world including in Nepal in the recent years with the declining physical, chemical and biological features of the soil. Therefore, this problem of declining soil fertility has been taken as the common global problem. Poudel said the World Soil Day is being marked with the main goal of raising extensive public awareness to tackle this growing problem of loss in soil fertility. The World Soil Day is being celebrated at the national level today in Nepal amidst various programmes under the aegis of the Department, Central Agricultural Laboratory, the National Soil Science Research Centre NARS, Food and Nutrition Security Improvement Project, Rural Enterprises and Economic Development Project, United Nations, Food and Agriculture Organization and the Nepalese Society of Soil Science. The UN has said that over the last 70 years, the level of vitamins and nutrients in food has drastically decreased, and it is estimated that 2 billion people worldwide suffer from lack of micronutrients, known as hidden hunger because it is difficult to detect. Soil degradation induces some soils to be nutrient depleted losing their capacity to support crops, while others have such a high nutrient concentration that represent a toxic environment to plants and animals, pollutes the environment and cause climate change. World Soil Day 2022 and its campaign 'Soils: Where food begins' aims to raise awareness of the importance of maintaining healthy ecosystems and human well-being by addressing the growing challenges in soil management, increasing soil awareness and encouraging societies to improve soil health.

Step 3: normalized text



View Topic (View Topic Extraction Process)

Step 1: Input Text

The world soil day programme reiterates the importance of soil for mankind and the crucial need for its conservation and proper management while at the same time increasing its fertility, the department of agriculture said. World soil day 2022 and its campaign soils where food begins aims to raise awareness of the importance of maintaining healthy ecosystems and human well-being by addressing the growing challenges in soil management, increasing soil awareness and encouraging societies to improve soil health.

Step 2: Preprocessed Text

the world soil day programme reiterates the importance of soil for mankind and the crucial need for its conservation and proper management while at the same time increasing its fertility the department of agriculture said world soil day and its campaign soils where food begins aims to raise awareness of the importance of maintaining healthy ecosystems and human well-being by addressing the growing challenges in soil management increasing soil awareness and encouraging societies to improve soil health

Step 3: Tokenized Words

the, world, soil, day, programme, reiterates, the, importance, of, soil, for, mankind, and, the, crucial, need, for, its, conservation, and, proper, management, while, at, the, same, time, increasing, its, fertility, the, department, of, agriculture, said, world, soil, day, and, its, campaign, soils, where, food, begins, aims, to, raise, awareness, of, the, importance, of, maintaining, healthy, ecosystems, and, human, well-being, by, addressing, the, growing, challenges, in, soil, management, increasing, soil, awareness, and, encouraging, societies, to, improve, soil, health

Step 4: Tagged Words

the, DT, world, NN, soil, NN, day, NN, programme, VB, reiterates, VBZ, the, DT, importance, NN, of, IN, soil, NN, for, IN, mankind, NN, and, CC, the, DT, crucial, JJ, need, NN, for, IN, its, PRPS, conservation, NN, and, CC, proper, JJ, management, NN, while, IN, at, IN, the, DT, same, JJ, time, NN, increasing, VBG, its, PRPS, fertility, NN, the, DT, department, NN, of, IN, agriculture, NN, said, VBD, world, NN, soil, NN, day, NN, and, CC, its, PRPS, campaign, NN, soils, VBZ, where, WRB, food, NN, begins, VBZ, aims, NNS, to, TO, raise, V, B, awareness, NN, of, IN, the, DT, importance, NN, of, IN, maintaining, VBG, healthy, JJ, ecosystems, NNS, and, CC, human, JJ, well-being, NN, by, IN, addressing, VBG, the, DT, growing, VBG, challenges, NNS, in, IN, soil, NN, management, NN, increasing, VBG, soil, NN, awareness, NN, and, CC, encouraging, JJ, societies, NNS, to, TO, improve, VB, soil, NN, health, NN

Step 5: Noun Words

world, soil, day, importance, soil, mankind, need, conservation, management, time, fertility, department, agriculture, world, soil, day, campaign, food, aims, awareness, importance, ecosystems, well-being, challenges, soil, management, soil, awareness, societies, soil, health

SOURCE CODE

1. LSA

```
#----- LSA implementation start -----

class LSASummarizer:

    def __init__(self, original_text):

        self.original_text = original_text

    def clean_text(self, text):

        """Clean the input text by removing special characters and digits."""

        clean_chars = []

        for char in text:

            if char.isalnum() or char.isspace() or char in ",|?=":

                clean_chars.append(char)

        clean_text = "".join(clean_chars).strip()

        clean_text = re.sub(r'\s+', ' ', clean_text) # Remove extra whitespace within text

        return clean_text

    def normalize_text(self, text):

        """Normalize text by applying advanced stemming and converting to lowercase."""

        return text.lower()

    def tokenize_sentences(self, text):

        """Split text into sentences based on common sentence ending punctuation."""

        sentences = []

        current_sentence = ""
```



```

for char in text:

    if char in ".!?:":

        sentences.append(current_sentence)

        current_sentence = ""

    else:

        current_sentence += char

sentences.append(current_sentence) # Add the last sentence

return sentences

```

```

def tokenize_words(self, sentences):

    """Split sentences into lists of words."""

    tokenized_sentences = []

    for sentence in sentences:

        words = sentence.split()

        tokenized_sentences.append(words)

    return tokenized_sentences

```

```

def create_term_document_matrix(self, tokenized_sentences):

    """Create a term-document matrix where rows represent words and columns
    represent sentences."""

    term_document_matrix = {}

    for sentence_index, sentence in enumerate(tokenized_sentences):

        for word in sentence:

            if word not in term_document_matrix:

                term_document_matrix[word] = [0] * len(tokenized_sentences)

```

```

        term_document_matrix[word][sentence_index] += 1

    return term_document_matrix

def calculate_dtm(self, term_document_matrix):
    """Transpose the term-document matrix to create the document-term matrix."""

    dtm = [[term_document_matrix[word][i] for word in term_document_matrix] for i in
range(len(term_document_matrix[next(iter(term_document_matrix))]))]

    return dtm

def mean_center(self, dtm):
    """Mean-center the document-term matrix."""

    for column in range(len(dtm[0])):

        column_mean = sum(row[column] for row in dtm) / len(dtm)

        for row in dtm:

            row[column] -= column_mean

    return dtm

def calculate_covariance_matrix_optimized(self, mean_centered_dtm):
    """Calculate the covariance matrix efficiently using NumPy."""

    dtm_array = np.array(mean_centered_dtm) # Convert to NumPy array for faster
operations

    covariance_matrix = np.cov(dtm_array.T) # Calculate covariance using NumPy's
optimized function

    return covariance_matrix

```

```

def perform_eigenvalue_decomposition(self, covariance_matrix):

    """Perform eigenvalue decomposition."""

    eigenvalues, eigenvectors = np.linalg.eig(covariance_matrix)

    sorted_indices = np.argsort(eigenvalues)[::-1]

    eigenvalues = eigenvalues[sorted_indices]

    eigenvectors = eigenvectors[:, sorted_indices]

    return eigenvalues, eigenvectors


def score_sentences(self, dtm, U_reduced, S_reduced):

    """Score sentences based on their importance."""

    sentence_scores = []

    for sentence_vector in dtm:

        concept_scores = np.dot(sentence_vector, U_reduced)

        weighted_scores = concept_scores * S_reduced

        sentence_score = np.sum(weighted_scores)

        sentence_scores.append(sentence_score)

    return sentence_scores


def select_top_sentences(self, sentence_scores, sentences, num_sentences=1):

    """Select top sentences based on their scores."""

    top_indices = np.argsort(sentence_scores)[-num_sentences:][::-1]

    selected_sentences = [sentences[i] for i in top_indices]

    return selected_sentences


def format_summary(self, sentences):

```

```
"""Format the summary by capitalizing each sentence and joining them into a paragraph."""
```

```
formatted_sentences = [sentence.strip().capitalize() for sentence in sentences]
```

```
formatted_paragraph = ". ".join(formatted_sentences)
```

```
return formatted_paragraph + '.'
```

```
def summarize(self, summary_length):
```

```
    """Generate LSA-based summary."""
```

```
    clean_text = self.clean_text(self.original_text)
```

```
    normalized_text = self.normalize_text(clean_text)
```

```
    sentences = self.tokenize_sentences(normalized_text)
```

```
    tokenized_sentences = self.tokenize_words(sentences)
```

```
    term_document_matrix = self.create_term_document_matrix(tokenized_sentences)
```

```
    dtm = self.calculate_dtm(term_document_matrix)
```

```
    mean_centered_dtm = self.mean_center(dtm)
```

```
    covariance_matrix =
```

```
self.calculate_covariance_matrix_optimized(mean_centered_dtm)
```

```
    eigenvalues, eigenvectors =
```

```
self.perform_eigenvalue_decomposition(covariance_matrix)
```

```
    U = np.column_stack(eigenvectors)
```

```
    S = np.diag(eigenvalues)
```

```
    Vt = U.T
```

```
    threshold = 0.1
```

```
    selected_components = np.where(eigenvalues >= threshold)[0]
```

```
    U_reduced = U[:, selected_components]
```

```

S_reduced = S[selected_components, selected_components]

Vt_reduced = Vt[:, selected_components]

sentence_scores = self.score_sentences(dtm, U_reduced, S_reduced)

summary_sentences = self.select_top_sentences(sentence_scores, sentences,
summary_length)

formatted_summary = self.format_summary(summary_sentences)

return formatted_summary

#-----LSA implementation end -----

```

2. Fine-Tuning TS Transformers

```

class CustomDataset(Dataset):

    def __init__(self, dataframe, tokenizer, source_len, summ_len):
        self.tokenizer = tokenizer
        self.data = dataframe
        self.source_len = source_len
        self.summ_len = summ_len
        self.text = self.data.abstract
        self.ctext = self.data.article

    def __len__(self):
        return len(self.text)

    def __getitem__(self, index):
        ctext = str(self.ctext[index])
        ctext = ' '.join(ctext.split())

        text = str(self.text[index])
        text = ' '.join(text.split())

```

```

        source = self.tokenizer.batch_encode_plus([cctx], max_length= self.source_len,
        pad_to_max_length=True,return_tensors='pt')

```

```

        target = self.tokenizer.batch_encode_plus([text], max_length= self.summ_len,
        pad_to_max_length=True,return_tensors='pt')

```

```

        source_ids = source['input_ids'].squeeze()
        source_mask = source['attention_mask'].squeeze()
        target_ids = target['input_ids'].squeeze()
        target_mask = target['attention_mask'].squeeze()

```

```

        return {
            'source_ids': source_ids.to(dtype=torch.long),
            'source_mask': source_mask.to(dtype=torch.long),
            'target_ids': target_ids.to(dtype=torch.long),
            'target_ids_y': target_ids.to(dtype=torch.long)
        }

```

Creating the training function.

```

def train(epoch, tokenizer, model, device, loader, optimizer):

```

```

    model.train()

```

```

    for _,data in enumerate(loader, 0):

```

```

        y = data['target_ids'].to(device, dtype = torch.long)

```

```

        y_ids = y[:, :-1].contiguous()

```

```

        lm_labels = y[:, 1:].clone().detach()

```

```

        lm_labels[y[:, 1:] == tokenizer.pad_token_id] = -100

```

```

        ids = data['source_ids'].to(device, dtype = torch.long)

```

```

        mask = data['source_mask'].to(device, dtype = torch.long)

```

```

        outputs = model(input_ids = ids, attention_mask = mask, decoder_input_ids=y_ids,
        labels=lm_labels)

```

```

        loss = outputs[0]

```

```

        if _%10 == 0:

```

```

wandb.log({"Training Loss": loss.item()})

if _%500==0:
    print(f'Epoch: {epoch}, Loss: {loss.item()}')

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
def validate(epoch, tokenizer, model, device, loader):
    model.eval()
    predictions = []
    actuals = []
    with torch.no_grad():
        for _, data in enumerate(loader, 0):
            y = data['target_ids'].to(device, dtype = torch.long)
            ids = data['source_ids'].to(device, dtype = torch.long)
            mask = data['source_mask'].to(device, dtype = torch.long)

            generated_ids = model.generate(
                input_ids = ids,
                attention_mask = mask,
                max_length=150
                num_beams=2,
                repetition_penalty=2.5,
                length_penalty=1.0,
                early_stopping=True
            )
            preds = [tokenizer.decode(g, skip_special_tokens=True,
clean_up_tokenization_spaces=True) for g in generated_ids]
            target = [tokenizer.decode(t, skip_special_tokens=True,
clean_up_tokenization_spaces=True)for t in y]
            if _%100==0:
                print(f'Completed {_}')

```

```
    predictions.extend(preds)
    actuals.extend(target)
return predictions, actuals
```

```
def main():
    wandb.init(project="mainsummarization")
    config = wandb.config
    config.TRAIN_BATCH_SIZE = 16
    config.VALID_BATCH_SIZE = 8
    config.TRAIN_EPOCHS = 3
    config.VAL_EPOCHS = 1
    config.LEARNING_RATE = 2e-5
    config.SEED = 42
    config.MAX_LEN = 256
    config.SUMMARY_LEN = 100

    # Set random seeds and deterministic pytorch for reproducibility
    torch.manual_seed(config.SEED) # pytorch random seed
    np.random.seed(config.SEED) # numpy random seed
    torch.backends.cudnn.deterministic = True

    # tokenizer for encoding the text
    tokenizer = T5Tokenizer.from_pretrained("t5-base")

    # Importing and Pre-Processing the domain data
    df = pd.read_csv('pubmed_35000_null_free.csv',encoding='latin-1')
    df = df[['article','abstract']]
    df.article = 'summarize: ' + df.article
    print(df.head())

    # Creation of Dataset and Dataloader
    train_size = 0.8
    train_dataset=df.sample(frac=train_size,random_state = config.SEED)
    val_dataset=df.drop(train_dataset.index).reset_index(drop=True)
```



```

train_dataset = train_dataset.reset_index(drop=True)

print("FULL Dataset: {}".format(df.shape))
print("TRAIN Dataset: {}".format(train_dataset.shape))
print("TEST Dataset: {}".format(val_dataset.shape))

# Creating the Training and Validation dataset for further creation of Dataloader
training_set = CustomDataset(train_dataset, tokenizer, config.MAX_LEN,
config.SUMMARY_LEN)
val_set = CustomDataset(val_dataset, tokenizer, config.MAX_LEN,
config.SUMMARY_LEN)

# Defining the parameters for creation of dataloaders
train_params = {
    'batch_size': config.TRAIN_BATCH_SIZE,
    'shuffle': True,
    'num_workers': 0
}

val_params = {
    'batch_size': config.VALID_BATCH_SIZE,
    'shuffle': False,
    'num_workers': 0
}

# Creation of Dataloaders for testing and validation. This will be used down for
training and validation stage for the model.
training_loader = DataLoader(training_set, **train_params)
val_loader = DataLoader(val_set, **val_params)

# Defining the model. We are using t5-base model and added a Language model layer
on top for generation of Summary.
# Further this model is sent to device (GPU/TPU) for using the hardware.

```

```

model_path = "pubmed_model"
model = T5ForConditionalGeneration.from_pretrained("t5-base")
model = model.to(device)

# Defining the optimizer that will be used to tune the weights of the network in the
training session.
optimizer = torch.optim.Adam(params = model.parameters(),
lr=config.LEARNING_RATE)
# Log metrics with wandb
wandb.watch(model, log="all")
# Training loop
print('Initiating Fine-Tuning for the model on our dataset')

for epoch in range(config.TRAIN_EPOCHS):
    train(epoch, tokenizer, model, device, training_loader, optimizer)
    model.save_pretrained(model_path)

# Validation loop and saving the resulting file with predictions and actuals in a
dataframe.
# Saving the dataframe as predictions.csv
print('Now generating summaries on our fine tuned model for the validation dataset
and saving it in a dataframe')
for epoch in range(config.VAL_EPOCHS):
    predictions, actuals = validate(epoch, tokenizer, model, device, val_loader)
    final_df = pd.DataFrame({'Generated Abstract':predictions,'Actual
Abstract':actuals})

# Save the CSV file with the specified name
csv_path = os.path.join(model_path, 'pubMedPredict.csv')
final_df.to_csv(csv_path, index=False)
print(f'Predictions saved at: {csv_path}')

if __name__ == '__main__':
    main()

```

Final Year Project Supervisor Meeting Log

Student Name/ TU Roll No. : Tilak Parajul/23282, Swechha Shrestha/23281, Suman khadka/23275

Project Title: Text Summarization Using LSA and T5 Transformers

Supervisor: Umesh Dahal

Supervision Date	Duration	Mode of Meeting	Observations	Supervisor Signature
25 December 2023	1 hour	Physical	Proposal defense	
5 February 2024	1 hour	Online google meet	Review on proposal and further work	
9 March 2024	1 hour	Online google meet	Mid Defense submission and Presentation	
12 March 2024	2 hours	Online google meet	Implementation review	
2 April 2024	1 hour	Online google meet	Presentation practice and feedback	
4 April 2024	1 hour	Online google meet	Discussion on further improvement for dashboard	